

Integrating Boomerang into TAGADA

Rocco Brunelli¹, Marine Minier², and Loïc Rouquette³

¹ Roma Tre University, Rome, Italy
`rocco.brunelli@uniroma3.it`

² Université de Lorraine, CNRS, Inria, LORIA, Nancy, France
`marine.minier@loria.fr`

³ EPITA Research Laboratory (LRE), 4/16 rue Voltaire, Le Kremlin-Bicêtre
F-94270, France
`loic.rouquette@epita.fr`

Abstract. Since 2009, the cryptographic community has its eyes fixed on automatic tools based on solvers to help the cryptanalysts trying to attack symmetric cryptographic schemes. Among those automatic tools, TAGADA is dedicated to search for a particular kind of cryptanalysis called differential cryptanalysis.

It is of major importance for the cryptographic community to have automatic tools dedicated to the analysis of security of symmetric key primitives to be convince about what symmetric key schemes should be used and what symmetric key schemes should not be used.

In this paper, we will see how to extend TAGADA from differential cryptanalysis to boomerang cryptanalysis which is an important kind of attacks in symmetric key cryptography. We will also compare our tool with the two existing ones dedicated to boomerang distinguishers proposed by Hadipour *et al.* and Derbez *et al.*

Keywords: Automatic tools, MILP, SAT, CP, differential cryptanalysis, boomerang cryptanalysis, block ciphers, TAGADA

Introduction

Since the seminal work of [29], the cryptographers try to use automatic tools to help solving cryptographic problems. Among these problems, the most promising approach seems to be to help cryptanalysts try to attack cryptographic schemes. Notably, in symmetric key cryptography, many solvers have been used to simplify the search of the best attacks regarding differential cryptanalysis [19,3], differential-linear cryptanalysis [21], boomerang attacks [16] or impossible attacks [5] ; even machine-learning has been used for this purpose [6]. So, it is of primary importance for the cryptanalytic community to continue to develop those tools to have automatic security evaluation of symmetric schemes and particularly block ciphers. Block ciphers are primitives from symmetric cryptography and are mainly used all over the world. For example, the AES standard [15] used in TLS is a block cipher.

In symmetric key cryptography, an attack, or cryptanalysis, is divided into two parts: the first one seeks to identify a distinguisher that occurs with a high probability, diverging significantly from the uniform distribution; the subsequent part uses this statistical distinguisher to formulate an attack, incorporating additional rounds at the beginning and/or end to partially recover some bits of the key. Therefore, it is essential to identify the optimal distinguisher, namely the one with the highest probability.

Two tools have been created to help symmetric key cryptanalysts to find block ciphers distinguishers: the first one is TAGADA [25] and the second one is CLAASP [4].

The first one uses a directed acyclic graph to represent the block cipher and is divided into two steps to search for differential distinguishers. This tool is efficient for what we call word-oriented block ciphers. Those particular block ciphers work on blocks where each block is divided into sub-blocks (say words) and the ciphering process is applied to each of this sub-block. TAGADA is divided into two Steps: the first one aims at abstracting each difference in each subblock with a Boolean value and we want to minimize the number of Boolean values equal to 1 whereas the second step, say Step 2, try to instantiate those Boolean values (*i.e.* the outputs of Step 1) into word values when maximizing the overall probability of the differential distinguishers.

The second CLAASP also represents a cryptographic primitive as a list of connected components in the form of a directed acyclic graph. From this representation, the library can automatically: (1) generate the Python or C code of the primitive evaluation function, (2) execute a wide range of statistical and avalanche tests on the primitive, (3) generate SAT, SMT, CP and MILP models to search, for example, differential and linear distinguishers, (4) measure algebraic properties of the primitive, (5) test neural-based distinguishers. CLAASP works for word-oriented ciphers but also bit-oriented ciphers.

Modelling boomerang attacks may be complex, time consuming and error prone. This paper suggests a new way to model and find automatically boomerang distinguishers on block ciphers. In this paper, we propose to extend the tool TAGADA originally dedicated to differential cryptanalysis on word-oriented block ciphers to boomerang cryptanalysis. To do so, we need to search no more 1 differential but two differentials, one in the encryption direction and one in the decryption direction mixing in the middle the two differentials. Moreover, and as TAGADA has two steps, we need to do this into each step with the same abstraction that the one used in TAGADA. As in TAGADA, the outputs of the Step 1 are the inputs of the Step 2.

This paper is organized as follows: Section 1 recalls preliminaries of cryptography to understand what is a block cipher, a differential cryptanalysis, a differential distinguisher and a boomerang distinguisher; Section 2 explains how TAGADA works and how to modify TAGADA to compute boomerang distin-

guishers; Section 3 compares our approach with existing ones (mainly the two existing ones [20,16]); Finally Section 4 concludes this paper.

1 Cryptographic Definitions

1.1 Block Ciphers and Word-Oriented Block Ciphers

A block cipher E_K is a function that takes as input a binary sequence K of size k bits, called key (also secret key or master key), a binary sequence M of size n bits, called plaintext or message, and returns a binary sequence C of size n bits, called ciphertext. More formally:

Definition 1 (Block Cipher).

$$E_K(M) = E(K, M) : \{0, 1\}^k \times \{0, 1\}^n \rightarrow \{0, 1\}^n \text{ and } C = E_K(M).$$

Block ciphers are in the family of symmetric ciphers. To do this, E_K should be associated with a decryption function, called D_K . This decryption function takes as input a K, C pair and returns M . More formally:

Definition 2 (Associated Deciphering function of a Block Cipher).

$$D_K(C) = D(K, C) = E_K^{-1}(C) : \{0, 1\}^k \times \{0, 1\}^n \rightarrow \{0, 1\}^n \text{ with} \\ M = D_K(C) \text{ and } \forall K, D_K(E_K(M)) = M.$$

In practice, it is difficult to directly evaluate the security of a whole encryption function. To maintain control over the security of the encryption function, cryptographers employ small, repeated round functions. An iterative block cipher refers to a cipher that uses this construction. Let N_r be the number of rounds of the encryption function and F the used round function; we have:

Definition 3 (Iterative Block Cipher).

$$E_K(M) = F_{K_{N_r-1}} \circ \dots \circ F_{K_1} \circ F_{K_0}(M)$$

We may notice that the round function F is using different subkeys $K_0, \dots, K_{N_r-1}$, called round subkeys. Those subkeys are derived from the master key K with a **key schedule** algorithm. Those round subkeys are necessary to avoid slide attacks [12].

Modern constructions of ciphers are predominantly grouped into three families: the **SPN** (**S**ubstitution **P**ermutation **N**etworks), the Feistel networks and the **ARX** (**A**dd **R**otate **X**or).

The two first families, which are SPN and Feistel networks, are mostly word oriented ciphers whereas ARX are mostly bit oriented ciphers. In the case of word-oriented ciphers, the primitives functions are defined over a Galois Field $\mathbb{F}_{2^m}$, where m is the size of the words. The most common used values for m are 8 (for bytes) and 4 (for nibbles). In the case of bit-oriented ciphers, the encryption is done using binary operations on large parts of the ciphertext. Our work is restricted to word-oriented ciphers.

1.2 Cryptanalysis

Differential Cryptanalysis In 1990, Biham and Shamir [9] introduced the notion of differential cryptanalysis, which made it possible to break the standard of the time: the DES [10].

The main idea of the differential cryptanalysis is to observe the behaviour of an encryption function when we disturb the plaintext. This involves observing the probability p of having an output difference δ_{out} when we have injected an input difference δ_{in} in the plaintext. In the case of a perfect cipher, such probability should be near to 2^{-n} . The pair $(\delta_{in}, \delta_{out})$ is called differential distinguisher if p is significantly⁴ higher than 2^{-n} .

Formally, the probability p is defined as:

$$p = \mathbb{P}_{E_K}(\delta_{out} \mid \delta_{in}) = \mathbb{P}(\delta_{out} = E_K(M) \oplus E_K(M \oplus \delta_{in}))$$

To improve the complexity of the differential attack, it is necessary to have an optimal distinguisher, *i.e.* a distinguisher with the highest probability. In practice, evaluating a differential distinguisher requires 2^{2n} computations. To simplify the search and the evaluation of differential distinguishers, we can use differential characteristics. A differential characteristic, noted $\delta_{in} \rightsquigarrow \delta_{out}$ is a path from δ_{in} to δ_{out} where all the intermediate differences are fixed. The probability of the differential is then the sum of the probabilities of all the differential characteristics it contains:

$$\mathbb{P}_{E_K}(\delta_{out} \mid \delta_{in}) = \sum \mathbb{P}(\delta_{in} \xrightarrow{E_K} \delta_{out})$$

A usual simplification is to consider the probability of the best differential characteristic, *i.e.* the one with the highest probability, as a good approximation of the probability of the differential:

$$\mathbb{P}_{E_K}(\delta_{out} \mid \delta_{in}) \simeq \max_{\delta_{in} \neq 0, \delta_{out} \neq 0} \mathbb{P}(\delta_{in} \xrightarrow{E_K} \delta_{out})$$

The evaluation of a differential characteristic can be easily done in iterative ciphers if we consider that each round is independent:

$$\mathbb{P}(\delta_{in} \xrightarrow{E_K} \delta_{out}) = \mathbb{P}(\delta_{in} \xrightarrow{F_{K_0}} \delta_{x_0}) \times \mathbb{P}(\delta_{x_0} \xrightarrow{F_{K_1}} \delta_{x_1}) \times \mathbb{P}(\delta_{x_1} \xrightarrow{F_{K_2}} \delta_{x_2}) \times \dots \times \mathbb{P}(\delta_{x_{N_r-2}} \xrightarrow{F_{K_{N_r-1}}} \delta_{out})$$

For each round $F_{K_{i-1}}$, the computation of the probability to go from the input difference $\delta_{x_{i-1}}$ to the output difference δ_{x_i} can be done by using the probability of its subfunctions. For those subfunctions, we have two categories:

- The functions that are linear in the Galois Field $\mathbb{F}_2^n$;
- The functions that are not linear in the Galois Field $\mathbb{F}_2^n$.

⁴ We consider that p is significantly higher than 2^{-n} if $p > 2^{-n} + \epsilon$. The choice of ϵ depends on the size of the plaintext.

In the case of linear functions, we have $f(x \oplus y) = f(x) \oplus f(y)$. As a result:

$$\begin{aligned}\mathbb{P}_f(\delta_{x_i} \mid \delta_{x_{i-1}}) &= \mathbb{P}(\delta_{x_i} = f(x) \oplus f(x \oplus \delta_{x_{i-1}})) \\ &= \mathbb{P}(\delta_{x_i} = f(\delta_{x_{i-1}}))\end{aligned}$$

This probability is one when $\delta_{x_i} = f(\delta_{x_{i-1}})$ and zero otherwise.

For a non-linear function (called NL here), in general, the round function of a block cipher is composed of many small non-linear components called S-boxes for saving hardware footprint. The current state of n bits, is decomposed into t words of m bits (usually $m = 4$ or 8) and each m bits word is passed in an S-box S . Then, we need to study the non-linear behaviour of S . So, in the case of a non-linear function S , the probability to go from the input difference δ_i to the output difference δ_o is given by the **DDT** (**Differential Distribution Table**) [9]:

Definition 4 (DDT).

$$DDT(\delta_{in}, \delta_{out}) = \frac{\#\{x \in \mathbb{F}_{2^n} \mid S(x) \oplus S(x \oplus \delta_i) = \delta_{out}\}}{2^n}$$

And we have $\mathbb{P}_S(\delta_{out} \mid \delta_{in}) = DDT(\delta_{in}, \delta_{out})$.

Remark 1. We may notice a special case when $\delta_{in} = 0$. In such case, $\delta_{out} = 0$ with a probability one since $S(x) \oplus S(x \oplus 0) = 0$.

Remark 2. As t S-boxes are in general called in parallel, we deduce that the whole non-linear layer is passed with a probability equal to the product of the probabilities of each DDT of each S-box: $\mathbb{P}_{NL} = \prod_{S \in NL} \mathbb{P}_S(\delta_{out} \mid \delta_{in})$.

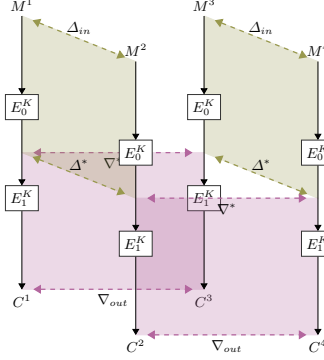
If we consider that the linear part of a block cipher round function can be traversed with a probability of one, it is the non-linear part of a block cipher round function that will determine the overall probability.

Once the probability computed, we have what we call a distinguisher, from this distinguisher we could mount attack by guessing some part of the subkeys involved when adding one or more rounds at the beginning and/or at the end [9]. Thus, the important step is to compute first the distinguisher.

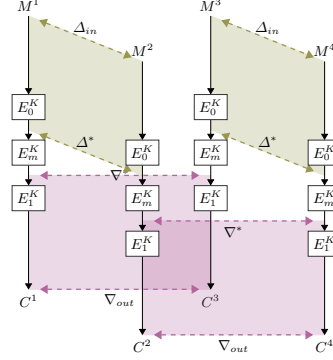
1.3 Boomerang Cryptanalysis

The differential attack could not be effective on some block ciphers when increasing the number of rounds [31]. Thus, in 1999, David Wagner in [32] introduced boomerang attacks. In this kind of attacks, an attacker tries to construct a distinguisher on a block cipher E where, after having ciphered a pair of messages with a difference Δ_{in} , the attacker shifts the ciphertexts from a difference ∇_{out} and hopes that the boomerang returns (see Figure 1 and Algorithm 7 in Appendix A), *i.e.* that the two decipherers have a difference Δ_{in} . In other words, the attacker waits for the relation

$$E^{-1}(E(M) \oplus \nabla_{out}) \oplus E^{-1}(E(M \oplus \Delta_{in}) \oplus \nabla_{out}) = \Delta_{in} \quad (1)$$



(a) Boomerang distinguisher.



(b) Sandwich distinguisher.

Fig. 1: Naive construction of a single-key boomerang distinguisher (left) and sandwich construction (right). $M^1 = M, M^2 = M \oplus \Delta_{in}, C^1 = E_K(M^1), C^2 = E_K(M^2), C^3 = C^1 \oplus \nabla_{out}$ and $C^4 = C^2 \oplus \nabla_{out}$.

that happens with a high probability, as far as possible from the uniform probability. At the origin, the attacker tries to combine a first differential of probability p that happens for the first rounds, say E_0 , and a second of probability q covering the final rounds, say E_1 . Thus, the cipher E is entirely covered by the distinguisher cut into two parts E_0 and E_1 ($E = E_1 \circ E_0$) with a probability equal to p^2q^2 .

Many generalisations have followed such as the related key boomerang framework [8], the rectangle attack [7] where data are only considered in the ciphering direction. But there are also many results of impossibility [26,23,11] that induced a full reconsideration of what is a Boomerang attack where the independence of the rounds could no more be considered. Then Dunkelman *et al.* [18] considered the sandwich framework where the cipher is cut into three (and no more two) parts $E = E_1 \circ E_m \circ E_0$ to capture what is happening in the middle, as shown in Figure 1b. Finally, the seminal work proposed at Eurocrypt 2018 [14] introduced the BCT framework to finer analyse E_m when it is composed of one SPN round. Many papers generalizing this approach to more rounds [34,28,35] or to different ciphers [13,33] have finally emerged. Then, the overall probability with these different methods become p^2q^2r where p^2 is the probability of E_0 , q is the probability of E_1 and r is the probability of E_m , the middle part, where the interaction of E_0 and E_1 is considered.

In the same time and with the recent emergence of automatic tools, several approaches have been proposed to automatically generated boomerang distinguishers, to cite few [16,20,22,24,27]. Among the previous results, there are two opposing views: in [16], the authors propose a complete framework where the exact probability (when possible) is exactly computed; in [20], the authors tried to compute the better possible E_0 and E_1 and experimentally evaluate the probability of E_m . This is this last way we decide to follow.

2 Modeling Boomerang Cryptanalysis with TAGADA

The aim of TAGADA is to model differential distinguishers for word-oriented block ciphers. TAGADA works in two steps: in Step 1, the differential values are abstracted by Boolean values where the Boolean value is one if there is a difference in the corresponding word and is 0 if there is no difference in the corresponding word. The aim of the Step 1 is to minimize the number of Boolean values equal to 1 entering S-boxes as the overall probability is entirely determined by S-boxes having a difference (we call those S-boxes active S-boxes). In this way, we are sure to have the characteristics with the minimal probability. The outputs (Step 1 could have many solutions) of Step 1 are the inputs of Step 2. Step 2 tries to instantiate with some true difference values the active S-boxes. When it is not possible, we say that we have a non-consistent solution. When it is possible, we have a consistent solution, and we try to maximize the overall probability of the differential distinguisher. This is our objective function for Step 2.

We will see, in this Section, how to extend TAGADA to boomerang distinguishers. In boomerang distinguishers, we no more look at a single differential characteristic but at two characteristics, one for E_0 (say the upper characteristic) and one for E_1 (say the lower characteristic) interacting in the middle part of the cipher E_m . We want also to maximize the overall probability of those three parts to maximize the overall probability of the whole boomerang distinguisher. So, this Section will first introduce the way TAGADA works; then describes how to cut the cipher into three parts and what have been done to tune Step 1 to search boomerang distinguisher; and finally describes how to modify Step 2 to maximize the probability of the overall boomerang distinguisher.

2.1 The TAGADA Library

The TAGADA library [25] use a bipartite graph based representation of ciphers (see Definition 5). In this representation each state or function is represented by a node and the edges represent the links between the variables and the functions. This graph representation of a cipher is called a *Specification Graph*. This representation also includes specific information that are related to the encryption process. This information is as follows: the plaintext P , the primary key K and the ciphertext C .

Definition 5 (Associated Specification Graph of a Block Cipher).

$SG = (SG_S, SG_F, SG_T, SG_P, SG_K, SG_C)$ where,

$$\begin{aligned}
SG_S &= \{s \mid s \text{ is a state}^5 \text{ of the cipher}\} \\
SG_F &= \{f \mid f \text{ is a function of the cipher}\} \\
SG_T &= \{((x_1, \dots, x_n), f, (y_1, \dots, y_m)) \in (SG_S^n) \times SG_F \times (SG_S^m) \mid (y_1, \dots, y_m) = f(x_1, \dots, x_n)\} \\
SG_P &= \{p \mid p \text{ is a plaintext word of the cipher}\} \\
SG_K &= \{k \mid k \text{ is a primary key word of the cipher}\} \\
SG_C &= \{c \mid c \text{ is a ciphertext word of the cipher}\}
\end{aligned}$$

For a given transition (or arc) $t \in SG_T$ such as $t = ((x_1, \dots, x_n), f, (y_1, \dots, y_m))$, we define its inputs as: $t_X = (x_1, \dots, x_n)$, its outputs as: $t_Y = (y_1, \dots, y_m)$ and its function as: $t_f = f$.

For a given state s , we define $\text{pred}(SG, s)$ the set of predecessors transitions of s . More formally: $\text{pred}(SG, s) = \{t \in SG_T, s \in t_Y\}$.

2.2 Modeling Boomerang Distinguishers: first steps

Splitting the block cipher E The computation of boomerang distinguishers can be done by using several techniques. In this paper we use the same construction as in [22]. The boomerang distinguisher is then represented by two differential characteristics, an upper characteristic, which represents the first part of the encryption process, and a lower characteristic, which represents the second part of the encryption process. The two characteristics are overlapping on a given number of rounds, in the middle, this section is called E_m (see Figure 2).

The probability of the boomerang distinguisher can be derived from the probability of both upper and lower characteristics with the boomerang approximation:

$$p^2 q^2 r. \quad (2)$$

To do so, we generate two characteristics, one characteristic for E_0 and E_m , called the upper characteristic, and one characteristic for E_m and E_1 called the lower characteristic.

Both E_0 and E_1 can be computed by usual differential models. The problem is E_m which links the two characteristics together in order to build the boomerang.

Creating E_0, E_m and E_1 automatically In order to build a valid boomerang distinguisher we need to split the cipher into three parts E_0, E_m and E_1 . In TAGADA, ciphers are described as graphs and the library does not handle "rounds". In order to cut the graph in three parts we rest on the variables names in order to retrieve the end of each parts. In TAGADA the input and

⁵ A state is either a plaintext word, a key word, an internal state (*i.e.* a temporary word created during the encryption process), a ciphertext word or a constant.

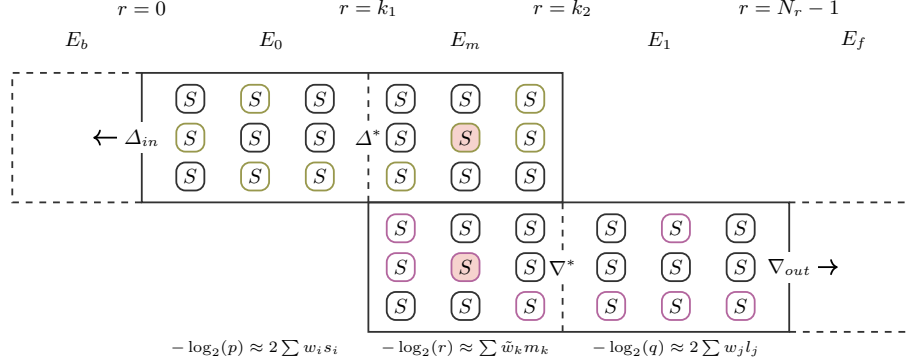


Fig. 2: Boomerang cryptanalysis construction. The upper characteristic is composed of three parts: E_b , E_0 and E_m . The lower characteristic is also composed of three parts: E_m , E_1 and E_f where the boomerang distinguisher is computed on E_0 , E_1 and E_m whereas the extended rounds for the attack are E_b and E_f . The cut of the three parts E_0 , E_m and E_1 are done on rounds k_1 and k_2 . The probability is approximated by evaluating the number of active S-Boxes in E_0 ($\sum s_i$), E_1 ($\sum l_j$) and the number of common active S-Boxes in E_m ($\sum m_k$). The weights w_i , w_j and w_k of those sums are defined in Section 2.3.

output of round functions are designated as X_r . By convention, X_r represents the set of output states for round r and, simultaneously, the set of input states for round $r+1$. As previously established in Figure 2, the final round of E_0 is k_1 , the final round of E_m is k_2 , and the final round of E_1 is $N_r - 1$. Consequently, the set of output states of E_0 is X_{k_1} , the set of output states E_m is X_{k_2} and the set of output states of E_1 is the ciphertext SG_C .

Having defined these output variables, the subsequent step involves retrieving the variables and transitions pertinent to each part. This is achieved by recursively defining the subgraph extraction process using Algorithm 1. For a given output variables O and a specified set of allowed transitions $T_{allowed}$, the algorithm identifies all predecessor states of O exclusively using the transitions within $T_{allowed}$.

The individual subgraphs are then formally defined as follows:

$$(S_{E_0}, F_{E_0}, T_{E_0}) = \text{extract_boomerang_part}(SG, SG_T, X_{k_1})$$

$$(S_{E_m}, F_{E_m}, T_{E_m}) = \text{extract_boomerang_part}(SG, SG_T \setminus T_{E_0}, X_{k_2})$$

$$(S_{E_1}, F_{E_1}, T_{E_1}) = \text{extract_boomerang_part}(SG, SG_T \setminus (T_{E_0} \cup T_{E_1}), SG_C)$$

Algorithm 1 `extract_boomerang_part`

Require: A Specification Graph SG , a set $T_{allowed}$ of allowed transitions and a set of output nodes O .

Ensure: A triplet (S, F, T) defined as follows: S is the set of nodes that can reach an output node in O using only transitions from the allowed set $T_{allowed}$. T is the minimal subset of $T_{allowed}$ that contains all the transitions necessary to connect the nodes in S to the output nodes in O . Finally, F is the set of functions utilized by the transitions in T .

```

1:  $S \leftarrow O$ 
2:  $T \leftarrow \emptyset$ 
3:  $F \leftarrow \emptyset$ 
4: updated  $\leftarrow$  true
5: while updated do
6:   updated  $\leftarrow$  false
7:   for each node  $s$  in  $S$  do
8:     for each transition  $t \in \text{pred}(SG, s)$  where  $t \in T_{allowed} \wedge t \notin T$  do
9:        $S \leftarrow S \cup t_I$ 
10:       $F \leftarrow F \cup \{t_f\}$ 
11:       $T \leftarrow T \cup \{t\}$ 
12:      updated  $\leftarrow$  true
13:    end for
14:  end for
15: end while
16: return  $(S, F, T)$ 

```

These definitions extract the subgraphs E_0 , E_m , and E_1 based on their respective output states (X_{k_1} , X_{k_2} , and SG_C) while successively restricting the set of available transitions (SG_T) to ensure the parts are disjoint in terms of transitions.

Remark 3. It is also pertinent to note that we can directly construct the concatenated subgraph E_0E_m by invoking the function: `extract_boomerang_part`(SG, SG_T, k_2). Similarly, the subgraph E_mE_1 can be constructed by calling `extract_boomerang_part`($SG, SG_T \setminus T_{E_0}, SG_C$) and then the subgraph E_m will be the intersection between those two subgraphs.

2.3 Step 1: Computing Truncated Differentials Distinguishers

We recall that the TAGADA Step 1 works on an abstraction where Boolean variables represent difference values. The aim of the Step 1 is to minimize the number of Boolean variables equal to 1 when entering the S-boxes.

Model the CP problem To model our CP problem we follow the general structure used to find truncated differential characteristics of [17] with several

adaptations inspired by [22]. First, we create the binary variables defining sets for the nodes in the E_0E_m , E_mE_1 and E_m graphs. As in [17] each variable is 1 if the state is active (i.e. it contains a difference) otherwise it is 0.

The objective is to minimize the function:

$$2 \underbrace{\sum_i w_i s_i}_{\text{Upper Prob: } -\log_2(p)} + 2 \underbrace{\sum_j w_j l_j}_{\text{Lower Prob: } -\log_2(q)} + \underbrace{\sum_k \tilde{w}_k m_k}_{\text{Middle Prob: } -\log_2(r)} \quad (3)$$

Let's analyze each term:

- $\sum_i w_i s_i$ contains the weights w_i that corresponds to the minus logarithm in base 2 of the maximum activation probability inside the DDT, $s_i \in \{0, 1\}$ are the variables related to the input state of an S-box in E_0 ; and it is multiplied by 2 since in the computation of the total probability of eq. (2) we have p^2 ;
- $\sum_j w_j l_j$ is equivalent to $\sum_i w_i s_i$ but for E_1 ;
- $\sum_k \tilde{w}_k m_k$ contains the weights $\tilde{w}_k$ that corresponds to the minus logarithm in base 2 of the maximum activation probability inside the BCT. $m_j \in \{0, 1\}$ are the variables related to the activation of an S-box in E_m . A specific condition is imposed for the activation of an S-Box within E_m . An S-Box in E_m is defined as active if and only if it is active in both the upper characteristic E_0E_m and in the lower characteristic E_mE_1 .

To model a boomerang, we firstly define two CSPs: one that reflects the E_0E_m characteristic and another that reflects the E_mE_1 characteristic. For the E_0E_m characteristic, we need to model the encryption process, while for the E_mE_1 characteristic, we need to model the decryption process. In this section, we will demonstrate how to model the E_0E_m characteristic and how to adapt the computation to model the E_mE_1 characteristic. Finally, we will show how to connect the middle part of the upper characteristic with the middle part of the lower characteristic and create the full CSP.

Modelling E_0E_m . The Constraint Satisfaction Problem (CSP) for the boomerang part E_0E_m is formally defined by the triplet (X, D, C) . In this standard formulation:

- The set of variables, X , is $\{x_i, \dots, x_n\}$,
- The set of domains, D , is $\{d_i, \dots, d_n\}$, which specifies the set of possible values for each variable x_i .
- The set C comprises the constraints imposed over the variables in X .

Given the triplet $(S_{E_0E_m}, F_{E_0E_m}, T_{E_0E_m})$ that represents the boomerang part E_0E_m , the set of variables X can be defined as the set of truncated differences of the states involved: $X = \{\Delta_s \mid s \in S_{E_0E_m}\}$. Here, Δ_s represents the truncated difference of state s . Consequently, the domain of each variable Δ_s is fixed to a binary set $\{0, 1\}$, reflecting the presence (1) or absence (0) of a difference in that state.

The set of constraints C is derived directly from the transition function $T_{E_0E_m}$. In the analysis of symmetric ciphers, the operators are broadly categorized into non-linear and linear components.

For the non-linear operators (*e.g.*, S-boxes), the constraint is governed by the property $S(x) \oplus S(x \oplus 0) = 0$. This formula signifies a key dependency: for any S-box application, the output difference of the S-box is active (non-zero) if and only if its corresponding input difference is also active (non-zero). This dependency is formally modeled by the following logical:

$$\begin{aligned} &\forall t \in T_{E_0E_m} \text{ where } t_f \text{ is an S-Box,} \\ &\text{let } (\Delta_{in}) = t_I \text{ and } (\Delta_{out}) = t_O, \quad \Delta_{in} = \Delta_{out} \end{aligned}$$

Linear Operator Linear operators are modeled as in [17]. For each linear operator f , an abstract table α_f is computed from the operator and posted as a constraint table. This abstract table is generated by brute-forcing all possible pairs of inputs, which limits the abstraction to 32-bit operators. More formally:

$$\begin{aligned} &\forall t \in T_{E_0E_m} \text{ where } t_f \text{ is linear,} \\ &\text{let } \alpha_{t_f} = \mathbf{abstract_table}(t_f), \quad t_I || t_O \in \alpha_{t_f}. \end{aligned}$$

To achieve the optimal differential characteristic for E_0E_m , we aim to select the solution with the highest probability. Consequently, the activation function is the one described in Equation 3.

$$\mathbf{minimize} \text{ } obj = 2 \sum_i w_i s_i$$

To remove the trivial solution which always occurs with no difference at all *-i.e.*, where two equivalent plaintexts will always be ciphered to the same ciphertext- we add an additional constraint that force at least one difference in the input. This difference may be either in the plaintext or in the master key for what we call related-key attacks, or only in the plaintext for single-key attacks.

$$\sum_{\Delta_s \in SG_P \cup SG_K} \Delta_s > 0$$

Modelling E_mE_1 . In the cipher E_mE_1 , the decryption direction must be modeled. However, since the encryption process is bijective, we can also model the cipher in the decryption direction. The objective function for E_mE_1 is also the one described in Equation 3.

$$\mathbf{minimize} \text{ } obj = 2 \sum_j w_j l_j$$

Here, we also want to eliminate the trivial solution that always results in no difference at all. However, for the lower trail, we want to ensure that there is at least a difference in the ciphertext or the key schedule for related-key attacks, or only in the ciphertext for single-key attacks.

$$\sum_{\Delta_s \in SG_C \cup SG_k} \Delta_s > 0$$

The calculation of key schedule states will be explained shortly.

Linking E_m . The segment E_m is analyzed in two distinct contexts: the end of the first path E_0E_m and the start of the second path E_mE_1 . To ensure path consistency and unify the two analyses, the respective Constraint Satisfaction Problems are merged. Furthermore, linking constraints are introduced to enforce correspondence between the two instances of E_m . Specifically, using the technique proposed in [22], the S-Boxes in the upper E_m instance are directly linked to the corresponding S-Boxes in the lower E_m instance. An S-Box within the shared E_m is defined as active if and only if both of its linked counterparts (the S-Boxes in the upper path and the S-Boxes in the lower path) are simultaneously active.

We defined the shared S-Box activity variable m for the segment E_m . For an S-Box $s^\uparrow$ in the upper trail and its corresponding lower S-Box $s^\downarrow$, the activity is constrained by the logical conjunction:

$$\forall s^\uparrow, s^\downarrow \in E_m : m = s^\uparrow \wedge s^\downarrow$$

This constraint enforces that the S-Box at this position in the shared segment is considered active only when both the upper and lower paths exhibit activity.

The goal is to determine the optimal differential characteristic by maximizing the expected probability of the boomerang return. This is achieved through the following objective function

$$\text{minimize } obj = \sum_k \tilde{w}_k m_k$$

where m_k are the S-Box activity variables and $\tilde{w}_k$ are weights derived from the BCT table.

Enhancing the Probability Evaluation of E_m . A significant distinction exists between the model presented in [1] (focused on SPN ciphers) and that of [2] (focused on Feistel Networks): the latter introduces additional constraints on the shared segment E_m .

These constraints are designed to enforce a probability-one transition (*i.e.*, preventing cancellations) across the S-Boxes in both the upper and lower differential trails. Formally, this mandates that once a word becomes active, it cannot

be deactivated by subsequent XOR operations involving other variables. Although the theoretical justification for this modification remains underexplored, the model incorporating these constraints yield probabilities that are empirically closer to the experimental evaluations.

A notable complexity in implementing this approach lies in its automation: the model requires a directional representation of the cipher, with E_m represented in the encryption direction for the upper path and the decryption direction for the lower path.

The differential propagation with probability-one constraints are manifested in two distinct ways, dependent on the type of cryptographic primitive considered. For S-Boxes: The constraints remain intrinsically tied to the characteristic of the S-box itself. Specifically, the activation of an output difference is directly conditioned upon the activation of a specific input difference. For Linear Operators: The propagation rule is more systematic. The generic formula dictates that if any input difference to the linear function is active (non-zero), all output differences resulting from that input will be considered active (non-zero). More formally:

$$\forall t \in T_{E_0 E_m} \text{ where } t_f \text{ is linear,}$$

$$\forall \Delta_o \in t_o, \Delta_o = \bigvee_{\Delta_i \in t_I} \Delta_i$$

The integration of the decryption function, which we model as the lower portion of the E_m graph, presents significant conceptual challenges. The primary difficulty lies in the fact that a simple inversion of the encryption graph is not universally applicable. Decryption strategies differ fundamentally according to the underlying cryptographic architecture, mainly between Substitution-Permutation Networks (SPNs) and Feistel networks.

In the case of SPNs, the design relies almost exclusively on the use of bijective functions (S-boxes, bit permutations). Consequently, obtaining the decryption function is conceptually straightforward: it requires the inversion of each component function (*e.g.*, using the inverse S-Box) and the reversal of the direction of the arcs in the data flow graph.

In contrast, Feistel networks do not require their round functions f to be bijective. Invertibility is an emergent property of the architecture itself. Let the round function of a Feistel network be defined as:

$$L_{i+1} = f(L_i, K_i) \oplus R_i$$

$$R_{i+1} = L_i$$

The decryption process is derived by rearranging the terms, without needing to invert f :

$$L_i = R_{i+1}$$

$$R_i = f(L_i, K_i) \oplus R_{i+1}$$

Beyond structural inversion, two major components of a cipher exhibit asymmetric behavior when transitioning to decryption.

The first challenge concerns the key schedule. Whether for encryption or decryption, this process is fundamentally unidirectional: it derives the round keys (derived keys) from the master key. The subgraph representing the key schedule must therefore not be inverted when modeling decryption; it is reused as is. Only the application of the round keys to the data path may follow a reversed order (*e.g.*, from RK_n to RK_1).

The second difficulty lies in the operation of adding the key to the text, usually performed via an exclusive-OR (XOR) operation. For encryption, we have:

$$C = P \oplus K$$

For decryption, the involutive property of XOR is used:

$$C \oplus K = P \tag{4}$$

This is not a functional inversion (XOR is its own inverse) but a reorganization of the operands. From a graphical modeling perspective, the operation node (XOR) is preserved, but its input connections (the ciphertext C and the key K) are reconfigured to produce the plaintext P . To correctly model the decryption process, a system must algorithmically differentiate between components. We employ the following formal strategies to: (1) detect the key schedule subgraph, (2) correctly identify and invert key-mixing operations, and (3) properly reverse the remaining data-path functions based on the cryptographic structure (*e.g.*, SPN or Feistel).

(1) We recursively define the set of elements belonging to the key schedule. An element is considered part of the key schedule if it is either an element of the primary key or is derived exclusively from other key schedule states. Formally, this property is captured by the predicate `is_key_schedule_state( $SG, s$ )`, whose definition relies on two supporting predicates.

A state s is a key schedule element if it is either a primary key element or if it is the output of a key schedule transition:

$$\begin{aligned} \text{is_key_schedule_state}(SG, s) = & \\ & \text{is_primary_key_element}(SG, s) \vee \\ & (\exists t \in \text{pred}(SG, s), \text{is_key_schedule_transition}(SG, t)) \end{aligned}$$

The supporting predicates are defined as follows:

- The predicate `is_primary_key_element( $SG, s$ )` identifies states that are part of the primary key, SG_K . This forms the base case for the recursion.

$$\text{is_primary_key_element}(SG, s) = s \in SG_K.$$

- The predicate $\text{is_key_schedule_transition}(SG, t)$ identifies transitions that belong to the key schedule. This holds if (1) all of its input states x are already key schedule elements, and (2) at least one input x is a variable state (to exclude operations that only involve constants).

$$\begin{aligned} \text{is_key_schedule_transition}(SG, t) = \\ (\forall x' \in t_X, \text{is_key_schedule_state}(SG, x')) \wedge (\exists x \in t_X, \text{is_variable}(SG, x)) \end{aligned}$$

Using these predicates, we define the Key Schedule associated with the specification graph SG as the subgraph KS .

Definition 6 (Key Schedule).

$$\begin{aligned} KS(SG) &= (S_{KS}, F_{KS}, T_{KS}) \\ S_{KS} &= \{s \in SG_S \mid \text{is_key_schedule_state}(SG, s)\} \\ F_{KS} &= \{f \in SG_F \mid \exists t \in T_{KS}, t_f = f\} \\ T_{KS} &= \{t \in SG_T \mid \text{is_key_schedule_transition}(SG, t)\} \end{aligned}$$

We define a Key Mixing transition as the step in which key material is integrated with the algorithm's internal data state. This is frequently implemented as an AddRoundKey operation. Formally, a transition t is a key-mixing transition if it has at least one input from the key schedule and at least one input from the data path.

(2) We define a Key Mixing transition as the step in which key material is integrated with the algorithm's internal data state. This is frequently implemented as an AddRoundKey operation. Formally, a transition t is a key-mixing transition if it has at least one input from the key schedule and at least one input from the data path.

$$\begin{aligned} \text{is_key_mixing_transition}(SG, t) = \\ (\exists x \in t_X, \text{is_variable}(SG, x) \wedge \text{is_key_schedule_state}(SG, x)) \wedge \\ (\exists x' \in t_X \setminus x, \text{is_variable}(SG, x') \wedge \neg \text{is_key_schedule_state}(SG, x')) \end{aligned}$$

(3) To generate the constraints for the decryption path, we do not explicitly compute the inverse graph. Instead, we traverse the graph SG and apply constraints based on the classification of each transition f .

- If f is a Key Schedule Transitions ($f \in F_{KS}$: The transition belongs to the keyschedule. The constraint is applied in the forward direction, identical to the encryption path, to preserve the unidirectionnal flow from the primary key to the derived rounds keys.

$$\forall y^L \in f_Y^L \quad y^L = \max_{x^L \in f_X^L} x^L$$

- if f is Invertible (Bijective): If the operator f is invertible and not a part of the key schedule (*e.g.*, an S-Box), the constraint for decryption is the inverse of the encryption path constraint.

$$\forall x^L \in f_X^L \quad x^L = \max_{y^L \in f_Y^L} y^L$$

- If f is a Key Mixing Transition: If f is a key mixing operator, it is typically an involution operation like XOR. We do not invert the function itself. Instead, we algebraically rearrange the constraint to solve the data-path input, given the key-path input and the output.

Let $f_{X_w}^L$ be the set of input variables that are known (computable without this transition) and $f_{X_u}^L$ be the set of input variables that are unknown (computable only with this transition), such that $f_X^L = f_{X_w}^L \cup f_{X_u}^L$ and $f_{X_w}^L \cap f_{X_u}^L = \emptyset$. For an XOR-based operation, we have:

$$\bigoplus_{x_w \in f_{X_w}^L} x_w \oplus \bigoplus_{x_u \in f_{X_u}^L} x_u = f_Y^L$$

We then rewrite the constraint to solve for the unknown inputs x_u :

$$\forall x_u \in f_{X_u}^L \quad x_u = \bigoplus_{x_w \in f_{X_w}^L} x_w \oplus \bigoplus_{y \in f_Y^L} y$$

For example, consider the round function of Warp:

$$s_{2i+1}^{r+1} = f(s_{2i+1}^r, k^r, s_{2i}^r) = s_{2i+1}^r \oplus k^r \oplus S(s_{2i}^r)$$

To generate the decryption constraint, we solve the data-path input s_{2i+1}^r :

$$s_{2i+1}^r = s_{2i+1}^{r+1} \oplus k^r \oplus S(s_{2i}^r).$$

Our analysis framework (TAGADA) does not explicitly store the inverse functions for operators. Therefore, the key idea of our algorithm is to start from the ciphertexts and traverse the graph visiting all nodes and adding the appropriate linear constraints during the process. The main challenge is to determine which nodes can be visited (*i.e.*, computed) at each step.

We initialize a Boolean flag for each variable, setting it to **true** for ciphertexts and key schedule states (which are considered known) and **false** for all the others. The cipher is considered "inverted" when all plaintext variables nodes are marked **true**. The pseudocode for this process is depicted in Algorithm 2.

The **inversion** algorithm (Algorithm 2) is guaranteed to terminate due to the invertibility of the encryption function (as per Definition 2), which ensures all nodes are reachable. The **can_be_evaluated** helper function in its basic form is described in Algorithm 3.

Algorithm 2 inversion

Require: Truncated Graph G , the set of ciphertext variables `ciphertext` and the set of key schedule elements `key_schedule_states`.

- 1: $C \leftarrow \text{ciphertext} \cup \text{key_schedule_states}$ $\triangleright$ The set of computable elements.
- 2: $T \leftarrow \text{transitions}(G)$
- 3: **while** T is not empty **do**
- 4: take t from T
- 5: $\text{is_evaluatable}, X_u, X_w, Y \leftarrow \text{can_be_evaluated}(t)$ $\triangleright$ See Algorithm (3)
- 6: **if** is_evaluatable **then**
- 7: **if** $t \in E_m$ and t_f is linear and t is not a key schedule transition **then**
- 8: Add constraints that model propagation with a probability of one.
- 9: **end if**
- 10: $C \leftarrow C \cup \{X_u\}$
- 11: remove t from T
- 12: **end if**
- 13: **end while**

Algorithm 3 can_be_evaluated

Require: A transition t , the set of variables that can already be calculated C .

- 1: $X_w \leftarrow \emptyset$
- 2: $X_u \leftarrow \emptyset$
- 3: **for** each input variable x in t_X **do**
- 4: **if** $x \in C$ **then**
- 5: $X_w \leftarrow X_w \cup \{x\}$
- 6: **else**
- 7: $X_u \leftarrow X_u \cup \{x\}$
- 8: **end if**
- 9: **end for**
- 10: $Y \leftarrow \emptyset$
- 11: **for** each output variable y in t_Y **do**
- 12: **if** $y \in C$ **then**
- 13: $Y \leftarrow \{y\}$
- 14: **else**
- 15: $X_u \leftarrow \{y\}$
- 16: **end if**
- 17: **end for**
- 18: $\text{is_evaluatable} \leftarrow (\#X_u = 1)$
- 19: **return** $\text{is_evaluatable}, X_u, X_w, Y$

The Output The output of the Step 1 is a file JSON that includes:

- the upper characteristic E_0E_m with the minus logarithm in base 2 of its truncated probability, called upper probability and computed by the active S-box in E_0 ;
- the lower characteristic E_mE_1 with the minus logarithm in base 2 of its truncated probability, called lower probability and computed by the active S-box in E_1 ;
- r the minus logarithm in base 2 of the E_m truncated probability, computed by the common active S-boxes in E_0E_m and E_mE_1 ;
- the minus logarithm in base 2 of the total probability (2).

Remark 4. Observe that the middle part in E_0E_m could be different from the one in E_mE_1 but the important part is to notice when both of them are simultaneously active.

2.4 Step 2: Computing Boomerang Probability

The probability evaluation of a boomerang attack, in its "sandwich" configuration, is based on the analysis of the three distinct components of the cipher, the initial part E_0 , the middle part E_m and the final part E_1 . The overall success probability of the attack is estimated using the formula : $P = p^2q^2r$, an approach detailed in [22]. The components of this formula are defined as follows:

- p represents the probability of the differential characteristic for the upper part, E_0 .
- q represents the probability of the differential characteristic for the lower part, E_1 .
- r is an experimentally evaluation that quantifies the probability of compatibility between the two differential characteristic through the middle part, E_m .

To compute this probability, we must consider several scenarios, depending on whether the components E_0 , E_m and E_1 are empty.

Exclusion of the Case Where E_m is empty. Within the strict framework of the "sandwich" attack, the middle component E_m must be non-empty by definition. A configuration where the size of E_m is null would not correspond to this attack model, but rather to earlier boomerang attack constructions. Such models are considered inadequate for our analysis. Therefore, we dismiss this case and require the middle part to be always non-empty.

Analysis of E_0 and E_1 Configurations. We distinguish three main cases based on the structure of the E_0 and E_1 parts.

Case 1: E_0 and E_1 are non-empty. This is the standard configuration for the sandwich attack. The analysis proceeds as follows:

1. Evaluation of p and q : We employ standard techniques of TAGADA for finding differential trails and the optimal differential characteristics for E_0 and E_1 . The associated probabilities p and q are then completely determined.
2. Experimental Estimation of r : The probability r is estimated empirically. More precisely, the output difference of the characteristic over E_0 and the input difference of the characteristic over E_1 are fixed. Experiments are conducted on E_m to count the number of valid pairs that verify the boomerang and thereby derive a robust estimate of r . The input difference of r is set to the output difference of E_0 and the output difference of r is set to the input difference of E_1 .

Case 2: E_0 and E_1 are empty. In this configuration, the attack is reduced to the analysis of the middle part E_m . The boomerang probability is therefore entirely contained within the probability r , while p and q are, by definition, equal to 1. Since the characteristics for E_0 and E_1 are empty, the input and output differences of E_m are unconstrained (in value). The estimation of r is thus performed by using randomly differences for the active input and output word differences.

Case 3: E_0 is empty or E_1 is empty. This asymmetric case occurs when one of the two outer parts is empty.

- If E_0 is empty (resp. E_1), its probability p (resp. q) is equal to 1. It is therefore not necessary to constraint the active input (resp. output) word differences of the middle part.
- The analysis focuses on finding an optimal differential characteristic for the non-empty part E_0 (resp. E_1), which gives us the probability p (resp. q).
- For the experimental estimation of r , since the part corresponding to the empty characteristic has no fixed differential values, random values are used for the active bits in the input (if E_0 is empty) or the output (if E_1 is empty) of the middle part E_m .

The details of the two sub-cases are given in Section B of the Appendix.

Step 2: Optimization This section details the optimization process for the second step of the boomerang attack. The primary task in Step 2 is to select concrete instantiations of the differential characteristics found for E_0 and E_1 . The optimization goal is to make the selection in a way that maximizes the final probability of the distinguisher, $P = p^2 q^2 r$.

The key of this optimization lies in the compatibility factor, r . While the probabilities p and q are fixed by the chosen characteristics, a single truncated characteristic can often be realized by several different concrete instantiations. Each instantiation can alter the input/output differences for the middle part, E_m , which directly impacts the value of r .

Our goal is to find the specific pair of instantiations for E_0 and E_1 that yields the highest possible r . However, exhaustively testing every combination is computationally infeasible due to the huge time requirements.

To solve this efficiently, we use a heuristic-based filtering algorithm (see Algorithm 5). This algorithm cleverly prunes the search space by identifying the most promising pairs of differences for E_m and by removing the pairs that lead to have the same E_m values. It operates using a score function, which ranks pairs based on their likelihood of producing a high r value.

This scoring approach is guided by:

- the "cluster effect" noted in [20]: difference pairs that appear frequently across many instantiations tend to be the most effective for the distinguisher and
- the "BCT effect" (see Appendix C)

By focusing our experimental evaluation only on these high-scoring pairs, we can efficiently find a near-optimal configuration.

It is worth noticing that the filter can be disabled by setting the score threshold to $\omega = -1$, which would cause the algorithm to evaluate all possible pairs.

The Output The output of the Step 2 is a file JSON that includes:

- the upper differential characteristic E_0 with the minus logarithm in base 2 of its probability p , called upper probability and computed by the active S-box in E_0 ;
- the lower differential characteristic E_1 with the minus logarithm in base 2 of its probability q , called lower probability and computed by the active S-box in E_1 ;
- r the minus logarithm in base 2 of the E_m truncated probability, computed by the distinguisher as described at the beginning of the Section (2.4);
- the minus logarithm in base 2 of the total probability (2).

Algorithm 4 UpdateSeen

Require: The set of all upper instantiations: $S_2^\uparrow$ and their associated truncated differential characteristic $s_1^\uparrow$, the set of all lower instantiations: $S_2^\downarrow$ and their associated truncated differential characteristic $s_1^\downarrow$, the set of already seen solutions: *seen*.

- 1: **if** $S_2^\uparrow \neq \emptyset$ and $S_2^\downarrow \neq \emptyset$ **then**
- 2: $seen \leftarrow seen \cup \{(s_1^\uparrow, s_1^\downarrow)\}$ ▷ We eliminate the couple.
- 3: **else if** $S_2^\uparrow \neq \emptyset$ and $S_2^\downarrow = \emptyset$ **then**
- 4: $seen \leftarrow seen \cup \{s_1^\downarrow\}$ ▷ We eliminate just the lower solutions.
- 5: **else if** $S_2^\uparrow = \emptyset$ and $S_2^\downarrow \neq \emptyset$ **then**
- 6: $seen \leftarrow seen \cup \{s_1^\uparrow\}$ ▷ We eliminate just the upper solutions.
- 7: **else**
- 8: $seen \leftarrow seen \cup \{s_1^\uparrow\}$
- 9: $seen \leftarrow seen \cup \{s_1^\downarrow\}$
- 10: **end if**
- 11: **return** *seen*

Algorithm 5 Filtering

Require: The set of all upper instantiations: $S_2^\uparrow$, the list of all lower instantiations $S_2^\downarrow$, the threshold probability: ω .

- 1: **function** UNIQUEBY(S, f)
Returns a new set containing only unique elements from S , based on the identifier extracted by function f .
- Require:** A set S , and a key extraction function $f : S \rightarrow ID$.
- Ensure:** A set $S' \subseteq S$ containing the unique elements of S with respect to the identifiers returned by f .
- 2: $S' \leftarrow \emptyset$ ▷ The set to store filtered unique items.
- 3: $ID_{seen} \leftarrow \emptyset$ ▷ The set of identifiers already encountered.
- 4: **for** each element $s \in S$ **do**
- 5: $id \leftarrow f(s)$
- 6: **if** $id \notin ID_{seen}$ **then**
- 7: $ID_{seen} \leftarrow ID_{seen} \cup \{id\}$
- 8: $S' \leftarrow S' \cup \{s\}$
- 9: **end if**
- 10: **end for**
- 11: **return** S'
- 12: **end function**
- 13: $S_2'^\uparrow \leftarrow \text{UNIQUEBY}(S_2^\uparrow, \text{output})$
- 14: $S_2'^\downarrow \leftarrow \text{UNIQUEBY}(S_2^\downarrow, \text{input})$
- 15: $result = \{(s^\uparrow, s^\downarrow) \in S_2'^\uparrow \times S_2'^\downarrow \mid \text{score}(s^\uparrow, s^\downarrow) \geq \omega\}$
- 16: **return** *result*

Algorithm 6 Tagada Boomerang Case1.

Require: The Specification Graph of the cipher: SG , the Truncated Differential Graph of the full cipher: ΔG , the Differential Graph of E_0 : $\delta G^\uparrow$ and the Differential Graph of E_1 : $\delta G^\downarrow$.

```

1:  $seen \leftarrow \{\}$ 
2:  $(p^2 q^2 r_{min}, p^2 q^2 r_{max}) \leftarrow (0, 1)$ 
3:  $(s_2^{\uparrow*}, s_2^{\downarrow*}) \leftarrow (None, None)$ 
4:  $s_1 \leftarrow \text{Step1}(\Delta G, seen, p^2 q^2 r_{max})$ 
5:  $p^2 q^2 r_{max} \leftarrow \text{TruncatedBoomerangProbability}(s_1)$ 
6: while  $p^2 q^2 r_{min} < p^2 q^2 r_{max}$  do
7:    $E_0 \leftarrow \text{UpperCharacteristic}(s_1)$ 
8:    $p, S_2^\uparrow \leftarrow \text{Step2}(\delta G^\uparrow, E_0)$   $\triangleright$  Returns all the possible differentials of  $E_0$  with the
      highest probability  $p$ .
9:    $E_1 \leftarrow \text{LowerCharacteristic}(s_1)$ 
10:   $q, S_2^\downarrow \leftarrow \text{Step2}(\delta G^\downarrow, E_1)$   $\triangleright$  Returns all the possible differentials of  $E_1$  with the
      highest probability  $q$ .
11:   $seen \leftarrow \text{UpdateSeen}(S_2^\uparrow, E_0, S_2^\downarrow, E_1, seen)$   $\triangleright$  See Algorithm (4)
12:   $S_2 \leftarrow \text{Filtering}(S_2^\uparrow, S_2^\downarrow, \alpha)$   $\triangleright$  See Algorithm (5)
13:   $(\bar{r}, s_2^{\uparrow}, s_2^{\downarrow}) \leftarrow (0, None, None)$ 
14:  for each pair  $(s_2'^{\uparrow}, s_2'^{\downarrow}) \in S_2$  do
15:     $\bar{r}' \leftarrow \text{EvaluateEm}(s_2'^{\uparrow}, s_2'^{\downarrow}, SG)$   $\triangleright$  See Algorithm (7)
16:    if  $p^2 \cdot q^2 \cdot \bar{r} < p^2 \cdot q^2 \cdot \bar{r}'$  then
17:       $(\bar{r}, s_2^{\uparrow}, s_2^{\downarrow}) \leftarrow (\bar{r}', s_2'^{\uparrow}, s_2'^{\downarrow})$ 
18:    end if
19:  end for
20:  if  $p^2 q^2 r_{min} < p^2 \cdot q^2 \cdot \bar{r}$  then
21:     $(p^2 q^2 r_{min}, s_2^{\uparrow*}, s_2^{\downarrow*}) \leftarrow (p^2 \cdot q^2 \cdot \bar{r}, s_2^{\uparrow}, s_2^{\downarrow})$ 
22:  end if
23:  if  $p^2 q^2 r_{min} < p^2 q^2 r_{max}$  then
24:     $s_1 \leftarrow \text{NextTruncatedDifferential}(\Delta G, seen, p^2 q^2 r_{max})$ 
25:    if  $s_1$  is None then
26:       $p^2 q^2 r_{max} \leftarrow \text{NextTheoreticalUpperBound}(\Delta G, p^2 q^2 r_{max})$ 
27:      if  $p^2 q^2 r_{min} < p^2 q^2 r_{max}$  then
28:        break
29:      end if
30:     $s_1 \leftarrow \text{Step1}(\Delta G, seen, p^2 q^2 r_{max})$ 
31:     $p^2 q^2 r_{max} \leftarrow \text{TruncatedBoomerangProbability}(s_1)$ 
32:  end if
33: end if
34: end while
35: return  $(s_2^{\uparrow*}, s_2^{\downarrow*}, p^2 q^2 r_{min})$ 

```

3 Results

There exist two major tools for computing boomerang distinguishers: the one⁶ by Hadipour *et al.* [20] (SPN ciphers) [22] (Feistel Ciphers), and the one⁷ by Derbez *et al.* [16]. For the dedicated case of WARP, another tool⁸ [24] is available and we use it for our comparison. In this Section, we compare our automatic tool with existing ones that are available on line. Our results for various ciphers are given in Table 1. Note that, as we could not exhibit the exact behaviour of what is happening in E_m , our results could differ from the ones of [20,16,24,22].

The way, we choose the boomerang inside E_m and the way we compute the "best" probability are empirical.

Note also that a boomerang distinguisher is pertinent as soon as its probability is under the size of the plaintext and/or the key (to mount the corresponding attack). This is why we only give results below this limit, except in the case of SKINNY-128 for 14 rounds, where we hope to reach the limit, but ultimately do not.

Application to WARP. WARP [1], proposed in 2020, is a lightweight cipher with 128 bits size for the plaintext and the key. It uses a 32-branch Generalized Feistel Structure (GFS) over 40 full rounds plus 1 partial round. The internal state is divided into 32 nibbles (X_i of 4 bits). The 128-bit master key is split into K^0 and K^1 , which are used alternately as the round key. A round applies an S-Box (32 parallel applications of the same S-box) and a round-key addition, followed by a permutation of the nibbles. Note that, for the case of WARP, our results beat the most recent state of the art [36].

Application to TWINE. TWINE [30] is a 64-bit block cipher (supporting 80- and 128-bit keys), proposed in 2012. It employs a Type-2 Generalized Feistel Structure with 16 4-bit branches (nibbles). It has 36 rounds in both key size variants. Each round consists of a nonlinear layer (8 parallel applications of the same S-Box) and a diffusion layer (a permutation of the 16 nibbles).

Application to SKINNY. SKINNY [2] is a family of very lightweight tweakable block ciphers, proposed in 2016 and standardised in 2022. It encrypts blocks of $n = 64$ or $n = 128$ bits, viewed as a 4×4 matrix of cells. The tweak (key + tweak) is $t = 64, 128, 192$ for $n = 64$ (or their double for $n = 128$) and is only XORed to the first two rows of the internal state. The round function allows a classical SPN structure: MixColumns uses a non-MDS binary matrix with a low weight. The number of rounds ranges from 32 to 56 depending on the block size to the tweak size ratio ($z = n/t$).

Cipher	Rnds	p	q	r	p^2q^2r	Reference	Time (in minutes)
SKINNY-64	11				$2^{-80.0}$ ($2^{-59.23}$)	[16]	161m †
	11	$2^{-8.00}$	$2^{-8.00}$	$2^{-24.00}$	$2^{-56.00}$	This paper	62m
SKINNY-128	13				2^{-122} ($2^{-112.53}$)	[16]	700m †
	13	$2^{-22.00}$	$2^{-22.00}$	$2^{-24.00}$	$2^{-112.00}$	This paper	713m
	14				2^{-158} ($2^{-128.52}$)	[16]	547m †
	14	$2^{-22.00}$	$2^{-42.00}$	$2^{-24.00}$	$2^{-152.00}$	This paper	3600m
TWINE	15	2^{-8}	2^{-8}	$2^{-19.03}$	$2^{-51.03}$	[22]	1m
	15	$2^{-4.00}$	$2^{-8.00}$	$2^{-24.00}$	$2^{-48.00}$	This paper	271m
	16	2^{-8}	2^{-8}	$2^{-26.04}$	$2^{-58.04}$	[22]	305m
	16	$2^{-8.00}$	$2^{-8.00}$	$2^{-24.00}$	$(2^{-56.00})^b$	This paper	371m
	16	$2^{-8.00}$	$2^{-8.00}$	$2^{-23.00}$	$(2^{-55.00})^a$	This paper	3180m
WARP	21	2^{-10}	2^{-19}	$2^{-26.55}$	$2^{-84.55}$	[20]	1m
					2^{-104}	[24]	1270m
	21	$2^{-14.00}$	$2^{-20.00}$	$2^{-23.00}$	$2^{-91.00}$	This paper	105m
	22	2^{-16}	2^{-19}	$2^{-26.55}$	$2^{-96.55}$	[20]	180m
					2^{-120}	[24]	3014m
	22	$2^{-16.00}$	$2^{-20.00}$	$2^{-24.00}$	$2^{-96.00}$	This paper	361m
	23	$2^{-24.00}$	$2^{-20.00}$	$2^{-27.59}$	$2^{-115.59}$	[20]	563m
	23	$2^{-16.00}$	$2^{-22.00}$	$2^{-24.00}$	$2^{-100.00}$	This paper	2400m

Table 1: Results for boomerang distinguishers for the three considered ciphers on Rnds rounds. The overall probability is given by p^2q^2r . The time is given in minutes. $(*)^a$ means that we have a score greater or equal than 0.1 while $(*)^b$ that the score is greater or equal than 0.5. For the results of [16], we provided two probabilities: (1) the probability computed without an extra step used to improve the probability accuracy and (2) the original paper results. The execution time with the † marker represents the time to compute the probability (1).

3.1 Summary

Computational Time All experiments were executed on the *Grid5000* “gros” cluster, using a node equipped with four **Intel Xeon Gold 6240L** CPUs (18 cores per CPU, 2.60 GHz). Unless otherwise specified, we used a single CPU (18 cores) for our experiments. Details of results and of computations could be found in Appendix D.

⁶ see <https://github.com/hadipourh/Boomerang> and <https://github.com/hadipourh/comeback>.

⁷ see <https://gitlab.inria.fr/pderbez/boomerangskinny>.

⁸ see <https://gitlab.inria.fr/lrouquet/boomerang-warp-fse-23>.

4 Conclusion

In this paper, we have seen how to extend TAGADA originally dedicated to differential distinguishers search to boomerang distinguishers search. We have compared this new tool with existing ones and seen that the results are quite encouraging.

It is of crucial importance for the symmetric cryptographic community to continue to develop those tools to have automatic bounds regarding the security of symmetric primitives. In the future, we plan to integrate the related-key variant into the boomerang distinguishers of TAGADA. In this variant, the differences are not only allowed in the plaintexts/ciphertexts but also in the key leading to four different keys to cipher/decipher the four messages that compose a boomerang.

In the future, we plan to integrate not only distinguishers to TAGADA but also whole attacks extending distinguishers with some rounds at the beginning and at the end to recover part of the secret key. This will allow to have tight security evaluation of symmetric primitives.

References

1. Banik, S., Bao, Z., Isobe, T., Kubo, H., Liu, F., Minematsu, K., Sakamoto, K., Shibata, N., Shigeri, M.: WARP : Revisiting GFN for lightweight 128-bit block cipher. In: Dunkelman, O., Jacobson, Jr., M.J., O’Flynn, C. (eds.) SAC 2020. LNCS, vol. 12804, pp. 535–564. Springer, Cham (Oct 2020). https://doi.org/10.1007/978-3-030-81652-0_21
2. Beierle, C., Jean, J., Kölbl, S., Leander, G., Moradi, A., Peyrin, T., Sasaki, Y., Sasdrich, P., Sim, S.M.: The SKINNY family of block ciphers and its low-latency variant MANTIS. In: Robshaw, M., Katz, J. (eds.) CRYPTO 2016, Part II. LNCS, vol. 9815, pp. 123–153. Springer, Berlin, Heidelberg (Aug 2016). https://doi.org/10.1007/978-3-662-53008-5_5
3. Bellini, E., De Piccoli, A., Formenti, M., G rault, D., Huynh, P., Pelizzola, S., Polese, S., Visconti, A.: Differential cryptanalysis with SAT, SMT, MILP, and CP: A detailed comparison for bit-oriented primitives. In: Deng, J., Kolesnikov, V., Schwarzmann, A.A. (eds.) CANS 23. LNCS, vol. 14342, pp. 268–292. Springer, Singapore (Oct / Nov 2023). https://doi.org/10.1007/978-981-99-7563-1_13
4. Bellini, E., G rault, D., Grados, J., Huang, Y.J., Makarim, R.H., Rachidi, M., Tiwari, S.K.: CLAASP: A cryptographic library for the automated analysis of symmetric primitives. In: Carlet, C., Mandal, K., Rijmen, V. (eds.) SAC 2023. LNCS, vol. 14201, pp. 387–408. Springer, Cham (Aug 2024). https://doi.org/10.1007/978-3-031-53368-6_19
5. Bellini, E., Huynh, P., G rault, D., Visconti, A., Piccoli, A.D., Pelizzola, S.: Impossible differential automation: Model generation and new techniques. IACR Cryptol. ePrint Arch. p. 1998 (2024), <https://eprint.iacr.org/2024/1998>
6. Benamira, A., G rault, D., Peyrin, T., Tan, Q.Q.: A deeper look at machine learning-based cryptanalysis. In: Canteaut, A., Standaert, F.X. (eds.) EURO-CRYPT 2021, Part I. LNCS, vol. 12696, pp. 805–835. Springer, Cham (Oct 2021). https://doi.org/10.1007/978-3-030-77870-5_28

7. Biham, E., Dunkelman, O., Keller, N.: The rectangle attack - rectangling the Serpent. In: Pfitzmann, B. (ed.) EUROCRYPT 2001. LNCS, vol. 2045, pp. 340–357. Springer, Berlin, Heidelberg (May 2001). https://doi.org/10.1007/3-540-44987-6_21
8. Biham, E., Dunkelman, O., Keller, N.: Related-key boomerang and rectangle attacks. In: Cramer, R. (ed.) EUROCRYPT 2005. LNCS, vol. 3494, pp. 507–525. Springer, Berlin, Heidelberg (May 2005). https://doi.org/10.1007/11426639_30
9. Biham, E., Shamir, A.: Differential cryptanalysis of DES-like cryptosystems. *Journal of Cryptology* **4**(1), 3–72 (Jan 1991). <https://doi.org/10.1007/BF00630563>
10. Biham, E., Shamir, A.: Differential cryptanalysis of the full 16-round DES. In: Brickell, E.F. (ed.) CRYPTO'92. LNCS, vol. 740, pp. 487–496. Springer, Berlin, Heidelberg (Aug 1993). https://doi.org/10.1007/3-540-48071-4_34
11. Biryukov, A., Khovratovich, D.: Related-key cryptanalysis of the full AES-192 and AES-256. In: Matsui, M. (ed.) ASIACRYPT 2009. LNCS, vol. 5912, pp. 1–18. Springer, Berlin, Heidelberg (Dec 2009). https://doi.org/10.1007/978-3-642-10366-7_1
12. Biryukov, A., Wagner, D.: Advanced slide attacks. In: Preneel, B. (ed.) EUROCRYPT 2000. LNCS, vol. 1807, pp. 589–606. Springer, Berlin, Heidelberg (May 2000). https://doi.org/10.1007/3-540-45539-6_41
13. Boukerrou, H., Huynh, P., Lallemand, V., Mandal, B., Minier, M.: On the Feistel counterpart of the boomerang connectivity table (long paper). *IACR Trans. Symm. Cryptol.* **2020**(1), 331–362 (2020). <https://doi.org/10.13154/tosc.v2020.i1.331-362>
14. Cid, C., Huang, T., Peyrin, T., Sasaki, Y., Song, L.: Boomerang connectivity table: A new cryptanalysis tool. In: Nielsen, J.B., Rijmen, V. (eds.) EUROCRYPT 2018, Part II. LNCS, vol. 10821, pp. 683–714. Springer, Cham (Apr / May 2018). https://doi.org/10.1007/978-3-319-78375-8_22
15. Daemen, J., Rijmen, V.: The design of Rijndael, vol. 2. Springer (2002)
16. Delaune, S., Derbez, P., Vavrille, M.: Catching the fastest boomerangs application to SKINNY. *IACR Trans. Symm. Cryptol.* **2020**(4), 104–129 (2020). <https://doi.org/10.46586/tosc.v2020.i4.104-129>
17. Delobel, F., Derbez, P., Gontier, A., Rouquette, L., Solnon, C.: A CP-based automatic tool for instantiating truncated differential characteristics - extended version. *Cryptology ePrint Archive*, Paper 2023/1831 (2023), <https://eprint.iacr.org/2023/1831>
18. Dunkelman, O., Keller, N., Shamir, A.: A practical-time related-key attack on the KASUMI cryptosystem used in GSM and 3G telephony. In: Rabin, T. (ed.) CRYPTO 2010. LNCS, vol. 6223, pp. 393–410. Springer, Berlin, Heidelberg (Aug 2010). https://doi.org/10.1007/978-3-642-14623-7_21
19. Gérault, D., Lafourcade, P., Minier, M., Solnon, C.: Computing AES related-key differential characteristics with constraint programming. *Artif. Intell.* **278** (2020). <https://doi.org/10.1016/J.ARTINT.2019.103183>, <https://doi.org/10.1016/j.artint.2019.103183>
20. Hadipour, H., Bagheri, N., Song, L.: Improved rectangle attacks on SKINNY and CRAFT. *IACR Trans. Symm. Cryptol.* **2021**(2), 140–198 (2021). <https://doi.org/10.46586/tosc.v2021.i2.140-198>
21. Hadipour, H., Derbez, P., Eichlseder, M.: Revisiting differential-linear attacks via a boomerang perspective with application to AES, ascon, CLEFIA, SKINNY, PRESENT, KNOT, TWINE, WARP, LBlock, simeck, and SERPENT. In: Reyzin, L., Stebila, D. (eds.) CRYPTO 2024, Part IV. LNCS, vol. 14923, pp. 38–72. Springer, Cham (Aug 2024). https://doi.org/10.1007/978-3-031-68385-5_2

22. Hadipour, H., Nageler, M., Eichlseder, M.: Throwing boomerangs into Feistel structures application to CLEFIA, WARP, LBlock, LBlock-s and TWINE. *IACR Trans. Symm. Cryptol.* **2022**(3), 271–302 (2022). <https://doi.org/10.46586/tosc.v2022.i3.271-302>
23. Kircanski, A.: Analysis of boomerang differential trails via a SAT-based constraint solver URSA. In: Malkin, T., Kolesnikov, V., Lewko, A.B., Polychronakis, M. (eds.) *ACNS 15 International Conference on Applied Cryptography and Network Security*. LNCS, vol. 9092, pp. 331–349. Springer, Cham (Jun 2015). https://doi.org/10.1007/978-3-319-28166-7_16
24. Lallemand, V., Minier, M., Rouquette, L.: Automatic search of rectangle attacks on Feistel ciphers: Application to WARP. *IACR Trans. Symm. Cryptol.* **2022**(2), 113–140 (2022). <https://doi.org/10.46586/tosc.v2022.i2.113-140>
25. Libralesso, L., Delobel, F., Lafourcade, P., Solnon, C.: Automatic generation of declarative models for differential cryptanalysis. In: Michel, L.D. (ed.) *27th International Conference on Principles and Practice of Constraint Programming, CP 2021, Montpellier, France (Virtual Conference), October 25-29, 2021*. LIPIcs, vol. 210, pp. 40:1–40:18. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2021). <https://doi.org/10.4230/LIPICS.CP.2021.40>, <https://doi.org/10.4230/LIPICS.CP.2021.40>
26. Murphy, S.: The return of the cryptographic boomerang. *IEEE Trans. Inf. Theory* **57**(4), 2517–2521 (2011). <https://doi.org/10.1109/TIT.2011.2111091>, <https://doi.org/10.1109/TIT.2011.2111091>
27. Rouquette, L., Minier, M., Solnon, C.: Automatic boomerang attacks search on rijndael. *J. Math. Cryptol.* **18**(1) (2024). <https://doi.org/10.1515/JMC-2023-0027>, <https://doi.org/10.1515/jmc-2023-0027>
28. Song, L., Qin, X., Hu, L.: Boomerang connectivity table revisited. *IACR Trans. Symm. Cryptol.* **2019**(1), 118–141 (2019). <https://doi.org/10.13154/tosc.v2019.i1.118-141>
29. Soos, M., Nohl, K., Castelluccia, C.: Extending SAT solvers to cryptographic problems. In: Kullmann, O. (ed.) *Theory and Applications of Satisfiability Testing - SAT 2009, 12th International Conference, SAT 2009, Swansea, UK, June 30 - July 3, 2009*. Proceedings. Lecture Notes in Computer Science, vol. 5584, pp. 244–257. Springer (2009). https://doi.org/10.1007/978-3-642-02777-2_24, https://doi.org/10.1007/978-3-642-02777-2_24
30. Suzuki, T., Minematsu, K., Morioka, S., Kobayashi, E.: TWINE : A lightweight block cipher for multiple platforms. In: Knudsen, L.R., Wu, H. (eds.) *SAC 2012*. LNCS, vol. 7707, pp. 339–354. Springer, Berlin, Heidelberg (Aug 2013). https://doi.org/10.1007/978-3-642-35999-6_22
31. Vaudenay, S.: Provable security for block ciphers by decorrelation. In: Morvan, M., Meinel, C., Krob, D. (eds.) *STACS 98, 15th Annual Symposium on Theoretical Aspects of Computer Science, Paris, France, February 25-27, 1998*, Proceedings. Lecture Notes in Computer Science, vol. 1373, pp. 249–275. Springer (1998). <https://doi.org/10.1007/BFb0028566>, <https://doi.org/10.1007/BFb0028566>
32. Wagner, D.: The boomerang attack. In: Knudsen, L.R. (ed.) *FSE’99*. LNCS, vol. 1636, pp. 156–170. Springer, Berlin, Heidelberg (Mar 1999). https://doi.org/10.1007/3-540-48519-8_12
33. Wang, D., Wang, B., Sun, S.: SAT-aided automatic search of boomerang distinguishers for ARX ciphers. *IACR Trans. Symm. Cryptol.* **2023**(1), 152–191 (2023). <https://doi.org/10.46586/tosc.v2023.i1.152-191>

34. Wang, H., Peyrin, T.: Boomerang switch in multiple rounds. IACR Trans. Symm. Cryptol. **2019**(1), 142–169 (2019). <https://doi.org/10.13154/tosc.v2019.i1.142-169>
35. Yang, Q., Song, L., Sun, S., Shi, D., Hu, L.: New properties of the double boomerang connectivity table. IACR Trans. Symm. Cryptol. **2022**(4), 208–242 (2022). <https://doi.org/10.46586/tosc.v2022.i4.208-242>
36. Zeng, X., Tan, L., Xu, H.: Improved search of boomerang distinguishers for generalized feistel and application to warp. IACR Transactions on Symmetric Cryptology **2025**(1), 444–470 (Mar 2025). <https://doi.org/10.46586/tosc.v2025.i1.444-470>, <https://tosc.iacr.org/index.php/ToSC/article/view/12084>

A Boomerang Distinguisher

Algorithm 7 gives the way the boomerang distinguisher works.

Algorithm 7 EvaluateEm

Require: The number of keys to try N_K , the number of quartets to try for each key N_Q , the input difference of E_m β , the output difference of E_m γ , and the partial cipher block E with it associated decryption function D .

```

1: counter  $\leftarrow 0$ 
2: repeat  $N_K$  times
3:    $K \leftarrow \text{random\_value}()$ 
4:   repeat  $N_Q$  times
5:      $M^1 \leftarrow \text{random\_value}()$ 
6:      $M^2 \leftarrow M^1 \oplus \alpha$ 
7:      $(C^1, C^2) \leftarrow (E_K(M^1), E_K(M^2))$ 
8:      $(C^3, C^4) \leftarrow (C^1 \oplus \beta, C^2 \oplus \beta)$ 
9:      $(M^3, M^4) \leftarrow (D_K(C^3), D_K(C^4))$ 
10:    if  $M^3 \oplus M^4 = \alpha$  then
11:      counter  $\leftarrow \text{counter} + 1$ 
12:    end if
13:  end repeat
14: end repeat
15: return  $-\log_2(\frac{\text{counter}}{N_K N_Q})$ 
```

B First case for Step 2

Case 3.A: E_0 is empty and E_1 is not empty.

In this case, we will have $p = 1$ but q will be the approximated probability of the Step 2 output from TAGADA (Algorithm 8). We are using in line 16 the function `Step1_next_possible_ub_lower` that is the computation of a new

upper bound in which we activate another S-box in the lower characteristic. For line 17, we have to move forward and at the same time: `p_boom` and `ub_lower`. Then, if we active one S-box in the lower characteristic this will correspond in a change of two times of the same quantity considering the total probability (remember that we have q^2). The output of Algorithm 8 gives us the input of the distinguisher: `upper_plaintext` that is the input of the truncated differential of E_0E_m (it will be Δ_{in}); the lower approximation of the probability q ; `Probability(s2_lower)`; and the lower differential E_1 `s2_lower` from which we extract the output of the distinguisher (i.e. it is the input in the encryption direction of E_1 that will be Δ_{out}).

Case 3.B: E_0 is empty and E_1 is not empty.

In this case. we will have $q = 1$ but p will be the approximated probability of the Step 2 output from TAGADA (Algorithm 9). We are using in line 16 the function `Step1_next_possible_ub_upper` that is the computation of a new upper bound in which we activate another S-box in the upper trail. For line 17, we have to move forward and at the same time: `p_boom` and `ub_upper`. Then, if we active one S-box in the upper characteristic this will correspond in a change of two times of the same quantity considering the total probability (remember that we have p^2). The output of Algorithm 9 gives us the output of the distinguisher: `lower_ciphertext` that is the output in the sense of the encryption direction of E_mE_1 (it will be Δ_{out}); the upper approximation of the probability is p ; `Probability(s2_upper)`; and the upper differential E_0 `s2_upper` from which we extract the input of the distinguisher (i.e. it is the output in the encryption direction of E_0 that will be Δ_{in}).

C Heuristic on the Optimization

As we mentioned before to analyze the probability of the propagation of the boomerang differentials, i.e. the boomerang switch, for one round in the middle, in 2018 it was introduced the BCT (Boomerang Connectivity Table) [14].

$$\text{BCT}(\Delta, \nabla) = \#\{x \in \mathbb{F}_{2^n} | S^{-1}(S(x) \oplus \nabla) \oplus S^{-1}(S(x \oplus \Delta) \oplus \nabla) = \Delta\}$$

After that, there were introduced other tables in the case of multiple rounds: the UBCT (Upper BCT) [34], the LBCT (Lower BCT) [28], the EBCT (Extended BCT) [16], and the FBCT (Feistel BCT) [13].

$$\text{FBCT}(\Delta, \nabla) = \#\{x \in \mathbb{F}_{2^n} | S(x) \oplus S(x \oplus \Delta) \oplus S(x \oplus \nabla) \oplus S(x \oplus \Delta \oplus \nabla) = 0\}$$

In Section 2.4, we introduced the filtering Algorithm 5 in which we didn't define the score function. Here, we have some heuristically proposals that reduce the computational time and find the same optimal solution. Supposing, that we have a state of n words than:

$$\text{score}(s^\uparrow, s^\downarrow) = \text{score}(\text{output}(s^\uparrow), \text{input}(s^\downarrow)) = \prod_{i=1}^n \prod_{j=1}^n f(\text{output}(s^\uparrow)_i, \text{input}(s^\downarrow)_j)$$

Algorithm 8 TAGADA Boomerang Case 3.A

Require: The Specification Graph of the cipher: SG , the Truncated Differential Graph of the full cipher: ΔG , the Differential Graph of E_0 : $\delta G^\uparrow$ and the Differential Graph of E_1 : $\delta G^\downarrow$.

```

1:  $seen \leftarrow \{\}$ 
2:  $(p^2 q^2 r_{min}, p^2 q^2 r_{max}) \leftarrow (0, 1)$ 
3:  $(E_0^*, s_2^{\downarrow*}) \leftarrow (None, None)$ 
4:  $s_1 \leftarrow \text{Step1}(\Delta G, seen, p^2 q^2 r_{max})$ 
5:  $p^2 q^2 r_{max} \leftarrow \text{TruncatedBoomerangProbability}(s_1)$ 
6:  $E_0 \leftarrow \text{UpperCharacteristic}(s_1)$ 
7: while  $p^2 q^2 r_{min} < p^2 q^2 r_{max}$  do
8:    $E_1 \leftarrow \text{LowerCharacteristic}(s_1)$ 
9:    $q, S_2^\downarrow \leftarrow \text{Step2}(\delta G^\downarrow, E_1) \triangleright$  Returns all the possible differentials of  $E_1$  with the
     highest probability  $q$ .
10:   $seen \leftarrow \text{UpdateSeen}(\{\}, E_0, S_2^\downarrow, E_1, seen) \triangleright$  See Algorithm (4)
11:   $S_2^{\downarrow*} \leftarrow \text{UniqueBy}(S_2^\downarrow, input) \triangleright$  See Algorithm (5)
12:   $(\bar{r}, s_2^{\downarrow}) \leftarrow (0, None)$ 
13:  for each  $s_2^{\downarrow} \in S_2^{\downarrow*}$  do
14:     $\bar{r}' \leftarrow \text{EvaluateEm}(output(E_0), s_2^{\downarrow}, SG) \triangleright$  See Algorithm (7)
15:    if  $p^2 \cdot q^2 \cdot \bar{r} < p^2 \cdot q^2 \cdot \bar{r}'$  then
16:       $(\bar{r}, s_2^{\downarrow}) \leftarrow (\bar{r}', s_2^{\downarrow})$ 
17:    end if
18:  end for
19:  if  $p^2 q^2 r_{min} < p^2 \cdot q^2 \cdot \bar{r}$  then
20:     $(p^2 q^2 r_{min}, E_0^*, s_2^{\downarrow*}) \leftarrow (p^2 \cdot q^2 \cdot \bar{r}, E_0, s_2^{\downarrow})$ 
21:  end if
22:  if  $p^2 q^2 r_{min} < p^2 q^2 r_{max}$  then
23:     $s_1 \leftarrow \text{NextTruncatedDifferential}(\Delta G, seen, p^2 q^2 r_{max})$ 
24:    if  $s_1$  is  $None$  then
25:       $p^2 q^2 r_{max} \leftarrow \text{NextTheoreticalUpperBound}(\Delta G, p^2 q^2 r_{max})$ 
26:      if  $p^2 q^2 r_{min} < p^2 q^2 r_{max}$  then
27:        break
28:      end if
29:       $s_1 \leftarrow \text{Step1}(\Delta G, seen, p^2 q^2 r_{max})$ 
30:       $p^2 q^2 r_{max} \leftarrow \text{TruncatedBoomerangProbability}(s_1)$ 
31:       $E_0 \leftarrow \text{UpperCharacteristic}(s_1)$ 
32:    end if
33:  end if
34: end while
35: return  $(E_0^*, s_2^{\downarrow*}, p^2 q^2 r_{min})$ 

```

Algorithm 9 TAGADA Boomerang Case 3.B

Require: The Specification Graph of the cipher: SG , the Truncated Differential Graph of the full cipher: ΔG , the Differential Graph of E_0 : $\delta G^\uparrow$ and the Differential Graph of E_1 : $\delta G^\downarrow$.

```

1:  $seen \leftarrow \{\}$ 
2:  $(p^2 q^2 r_{min}, p^2 q^2 r_{max}) \leftarrow (0, 1)$ 
3:  $(s_2^{\uparrow*}, E_1^*) \leftarrow (None, None)$ 
4:  $s_1 \leftarrow \text{Step1}(\Delta G, seen, p^2 q^2 r_{max})$ 
5:  $p^2 q^2 r_{max} \leftarrow \text{TruncatedBoomerangProbability}(s_1)$ 
6:  $E_1 \leftarrow \text{LowerCharacteristic}(s_1)$ 
7: while  $p^2 q^2 r_{min} < p^2 q^2 r_{max}$  do
8:    $E_0 \leftarrow \text{UpperCharacteristic}(s_1)$ 
9:    $p, S_2^\uparrow \leftarrow \text{Step2}(\delta G^\uparrow, E_0)$   $\triangleright$  Returns all the possible differentials of  $E_0$  with the
      highest probability  $p$ .
10:   $seen \leftarrow \text{UpdateSeen}(S_2^\uparrow, E_0, \{\}, E_1, seen)$   $\triangleright$  See Algorithm (4)
11:   $S_2^{\prime\uparrow} \leftarrow \text{UniqueBy}(S_2^\uparrow, output)$   $\triangleright$  See Algorithm (5)
12:   $(\bar{r}, s_2^{\uparrow}) \leftarrow (0, None)$ 
13:  for each  $s_2^{\prime\uparrow} \in S_2^{\prime\uparrow}$  do
14:     $\bar{r}' \leftarrow \text{EvaluateEm}(s_2^{\prime\uparrow}, input(E_1), SG)$   $\triangleright$  See Algorithm (7)
15:    if  $p^2 \cdot q^2 \cdot \bar{r} < p^2 \cdot q^2 \cdot \bar{r}'$  then
16:       $(\bar{r}, s_2^{\uparrow}) \leftarrow (\bar{r}', s_2^{\prime\uparrow})$ 
17:    end if
18:  end for
19:  if  $p^2 q^2 r_{min} < p^2 \cdot q^2 \cdot \bar{r}$  then
20:     $(p^2 q^2 r_{min}, s_2^{\uparrow*}, E_1^*) \leftarrow (p^2 \cdot q^2 \cdot \bar{r}, s_2^{\uparrow}, E_1)$ 
21:  end if
22:  if  $p^2 q^2 r_{min} < p^2 q^2 r_{max}$  then
23:     $s_1 \leftarrow \text{NextTruncatedDifferential}(\Delta G, seen, p^2 q^2 r_{max})$ 
24:    if  $s_1$  is None then
25:       $p^2 q^2 r_{max} \leftarrow \text{NextTheoreticalUpperBound}(\Delta G, p^2 q^2 r_{max})$ 
26:      if  $p^2 q^2 r_{min} < p^2 q^2 r_{max}$  then
27:        break
28:      end if
29:       $s_1 \leftarrow \text{Step1}(\Delta G, seen, p^2 q^2 r_{max})$ 
30:       $p^2 q^2 r_{max} \leftarrow \text{TruncatedBoomerangProbability}(s_1)$ 
31:       $E_1 \leftarrow \text{LowerCharacteristic}(s_1)$ 
32:    end if
33:  end if
34: end while
35: return  $(s_2^{\uparrow*}, E_1^*, p^2 q^2 r_{min})$ 

```


For the WARP and TWINE cipher we set $f(x, y) = FBCT(x, y)$, while for SKINNY $f(x, y) = BCT(x, y)$. Then, we keep the pair with the score greater of a certain threshold α i.e. $f(x, y) \geq \alpha$. If α is set to 0 then we are not applying any extra filtering. For TWINE see Table 3, for SKINNY64 see Table 5, for SKINNY128 see Table 6 and for WARP see 8.

Remark 5. The BCT and the FBCT are applied when the number of rounds of the middle part is 1. In the case of several rounds, the EBCT, UBCT and LBCT are usually applied at every step of the middle part. Since we are not instantiating the E_m part, for either the upper and the lower part we cannot apply those tables.

D Complete Results for TWINE, WARP and SKINNY

Table 2 gives our complete results concerning the block cipher TWINE. Table 4 gives our complete results concerning the block cipher SKINNY. Table 7 gives our complete results concerning the block cipher WARP. Note also that, in the tool provided in [16], there are two ways to compute the boomerang probability: one simple without any computation refinement for E_m and one with a refinement for the computation of E_m called here "Sandwich probability".

Cipher	Rounds	Boom. Proba.	Time
TWINE	13	$(2^{-31.69})^a$	21h 37m
		$(2^{-31.76})^b$	4h 53m
		$2^{-34.32}$ [20]	-
	14	$(2^{-40.00})^a$	1day 8h
		$(2^{-39.81})^b$	9h 3m
		$2^{-42.52}$ [20]	-
	15	$(2^{-48.00})^a$	1day 3h
		$(2^{-48.00})^b$	4h 31m
		$2^{-51.03}$ [20]	-
	16	$(2^{-55.00})^a$	2days 5h
		$(2^{-56.00})^b$	6h 11m
		$2^{-58.04}$ [20]	-
	17	$(2^{-68.00})^a$	19h 55m
		$(2^{-68.00})^b$	6h 11m
		- [20]	-
	18	$(2^{-80.00})^a$	1h 28s
		$(2^{-79.00})^b$	8m 33s
		- [20]	-
	19	$2^{-84.00}$	7h 40m
		- [20]	-

Table 2: Boomerang distinguishers with their corresponding probability and number of rounds for TWINE. The computation time is given in hours, minutes and seconds. $(*)^a$ means that we have the score greater or equal than 0.1 while $(*)^b$ that the score is greater or equal than 0.5.

E Difference for TWINE, WARP and SKINNY

To denote the input difference of round $r + 1$ in upper and lower trails of the boomerang attack, we use the notation of [20], $\Delta X^{(r)}$ and $\nabla X^{(r)}$. In addition, we use $\Delta X_i^{(r)}$ (or $\nabla X_i^{(r)}$) to denote the difference of the i -th nibble in the input of round $r + 1$. Table 9 gives our complete results concerning the block cipher TWINE. Tables 10 and 11 give our complete results concerning the block cipher SKINNY. Tables 12 and 13 give our complete results concerning the block cipher WARP.

Cipher	Rnds	p	q	r	p^2q^2r	α	Time (in minutes)
TWINE	13	—	—	—	—	0	3600.00m
	13	$2^{-4.00}$	$2^{-4.00}$	$2^{-15.69}$	$2^{-31.69}$	0.1	1297.00m
	13	$2^{-4.00}$	$2^{-4.00}$	$2^{-15.76}$	$2^{-31.76}$	0.5	293.00m
	14	—	—	—	—	0	3600.00m
	14	$2^{-4.00}$	$2^{-4.00}$	$2^{-24.00}$	$2^{-40.00}$	0.1	1920.00m
	14	$2^{-4.00}$	$2^{-8.00}$	$2^{-15.81}$	$2^{-39.81}$	0.5	543.00m
	15	—	—	—	—	0	3600.00m
	15	$2^{-4.00}$	$2^{-8.00}$	$2^{-24.00}$	$2^{-48.00}$	0.1	1620.00m
	15	$2^{-4.00}$	$2^{-8.00}$	$2^{-24.00}$	$2^{-48.00}$	0.5	271.00m
	16	—	—	—	—	0	3600.00m
	16	$2^{-8.00}$	$2^{-8.00}$	$2^{-23.00}$	$2^{-55.00}$	0.1	3180.00m
	16	$2^{-8.00}$	$2^{-8.00}$	$2^{-24.00}$	$2^{-56.00}$	0.5	371.00m
	17	—	—	—	—	0	3600.00m
	17	$2^{-14.00}$	$2^{-8.00}$	$2^{-24.00}$	$2^{-68.00}$	0.1	1195.00m
	17	$2^{-14.00}$	$2^{-8.00}$	$2^{-24.00}$	$2^{-68.00}$	0.5	371.00m
	18	$2^{-10.00}$	$2^{-14.00}$	$2^{-19.14}$	$2^{-67.14}$	0	115.67m
	18	$2^{-14.00}$	$2^{-14.00}$	$2^{-24.00}$	$2^{-80.00}$	0.1	60.47m
	18	$2^{-14.00}$	$2^{-14.00}$	$2^{-23.00}$	$2^{-79.00}$	0.5	8.55m
	19	$2^{-14.00}$	$2^{-16.00}$	$2^{-24.00}$	$2^{-84.00}$	0	460.00m
	19	$2^{-14.00}$	$2^{-24.00}$	$2^{-23.00}$	$2^{-99.00}$	0.1	1332.00m
	19	$2^{-14.00}$	$2^{-24.00}$	$2^{-24.00}$	$2^{-100.00}$	0.5	396.00m

Table 3: Results for TWINE

Cipher	Round	Boom Prob	Time
SKINNY64	9	$2^{-33.44}$	1h 54s
		2^{-46} [16]	36s
	10	$2^{-44.00}$	1h 3m 12s
		2^{-61} [16]	1m 14s
	11	$2^{-56.00}$	1h 2m 10s
		$2^{-59.23}$ [16] + Sandwich Proba	2m 41s (without Sandwich Proba)
SKINNY128	12	$2^{-76.00}$	21h 4m
		2^{-102} [16]	13m 57s
	10	$2^{-44.00}$	5h 59m
		2^{-64} [16]	1m 17s
	11	$2^{-56.00}$	5h 59m
		2^{80} [16]	3m 38s
	12	$2^{-84.00}$	5h 54m
		2^{-104} [16]	25m 28s
	13	$2^{-112.00}$	11h 53m
		$2^{-112.53}$ [16] + Sandwich Proba	11m 46s (without Sandwich Proba)
	14	$2^{-152.00}$	60h
		$2^{-128.52}$ [16] + Sandwich Proba	9m 08s

Table 4: Boomerang distinguishers with their corresponding probability and number of rounds for SKINNY. The computation time is given in hours, minutes and seconds. $(*)^a$ means that we have the score greater or equal than 0.1 while $(*)^b$ that the score is greater or equal than 0.5.

Cipher	Rnds	p	q	r	p^2q^2r	α	Time (in minutes)
SKINNY64	9	$2^{-6.00}$	$2^{-2.00}$	$2^{-17.44}$	$2^{-33.44}$	0	60.90m
	9	$2^{-6.00}$	$2^{-2.00}$	$2^{-17.44}$	$2^{-33.44}$	0.1	35.80m
	9	$2^{-6.00}$	$2^{-2.00}$	$2^{-17.37}$	$2^{-33.37}$	0.5	8.83m
	10	$2^{-8.00}$	$2^{-2.00}$	$2^{-24.00}$	$2^{-44.00}$	0	63.20m
	10	$2^{-8.00}$	$2^{-2.00}$	$2^{-24.00}$	$2^{-44.00}$	0.1	34.55m
	10	$2^{-8.00}$	$2^{-2.00}$	$2^{-24.00}$	$2^{-44.00}$	0.5	8.30m
	11	$2^{-8.00}$	$2^{-8.00}$	$2^{-24.00}$	$2^{-56.00}$	0	62.17m
	11	$2^{-8.00}$	$2^{-8.00}$	$2^{-24.00}$	$2^{-56.00}$	0.1	35.83m
	11	$2^{-8.00}$	$2^{-8.00}$	$2^{-24.00}$	$2^{-56.00}$	0.5	8.37m
	12	$2^{-18.00}$	$2^{-8.00}$	$2^{-24.00}$	$2^{-76.00}$	0	1264.00m
	12	$2^{-18.00}$	$2^{-8.00}$	$2^{-24.00}$	$2^{-76.00}$	0.1	471.00m
	12	—	—	—	—	0.5	3600.00m

Table 5: Results for SKINNY64

Cipher	Rnds	p	q	r	p^2q^2r	α	Time (in minutes)
SKINNY128	10	$2^{-8.00}$	$2^{-2.00}$	$2^{-24.00}$	$2^{-44.00}$	0	359.00m
	10	$2^{-8.00}$	$2^{-2.00}$	$2^{-24.00}$	$2^{-44.00}$	0.1	250.00m
	10	$2^{-8.00}$	$2^{-2.00}$	$2^{-10.31}$	$2^{-30.31}$	0.5	118.67m
	11	$2^{-8.00}$	$2^{-8.00}$	$2^{-24.00}$	$2^{-56.00}$	0	359.00m
	11	$2^{-8.00}$	$2^{-8.00}$	$2^{-24.00}$	$2^{-56.00}$	0.1	252.00m
	11	$2^{-8.00}$	$2^{-8.00}$	$2^{-24.00}$	$2^{-56.00}$	0.5	58.28m
	12	$2^{-22.00}$	$2^{-8.00}$	$2^{-24.00}$	$2^{-84.00}$	0	354.00m
	12	$2^{-22.00}$	$2^{-8.00}$	$2^{-24.00}$	$2^{-84.00}$	0.1	202.00m
	12	$2^{-22.00}$	$2^{-8.00}$	$2^{-24.00}$	$2^{-84.00}$	0.5	105.23m
	13	$2^{-22.00}$	$2^{-22.00}$	$2^{-24.00}$	$2^{-112.00}$	0	713.00m
	13	$2^{-22.00}$	$2^{-22.00}$	$2^{-24.00}$	$2^{-112.00}$	0.1	321.00m
	13	$2^{-22.00}$	$2^{-22.00}$	$2^{-24.00}$	$2^{-112.00}$	0.5	105.13m
	14	—	—	—	—	0	3600.00m
	14	—	—	—	—	0.1	3600.00m
	14	—	—	—	—	0.5	3600.00m
	14	$2^{-22.00}$	$2^{-42.00}$	$2^{-24.00}$	$2^{-152.00}$	0.8	1560.00m
	14	$2^{-22.00}$	$2^{-42.00}$	$2^{-24.00}$	$2^{-152.00}$	1.0	2040.00m

Table 6: Results for SKINNY128

Cipher	Round	Boom Prob	Time
WARP	14	$2^{-20.55}$ $2^{-20.58}$ [20] 2^{-20} [24]	1h 43m 51s 4s 1m 35s
	15	$2^{-26.50}$ $2^{-28.58}$ [20] 2^{-28} [24]	1h 45m 46s 8s 2m 04s
	16	$2^{-36.51}$ $2^{-34.50}$ [20] 2^{-38} [24]	1h 44m 55s 12s 3m 25s
	17	$2^{-42.51}$ - [20] 2^{-46} [24]	1h 45m 59s - 9m 12s
	18	$2^{-52.00}$ - [20] 2^{-58} [24]	1h 44m 9s - 22m 59s
	19	$2^{-64.19}$ - [20] 2^{-70} [24]	1h 45m 11s - 1h 01m 07s
	20	$2^{-76.19}$ $2^{-75.96}$ [20] 2^{-84} [24]	1h 47m 30s 31s 3h 01m 39s
	21	$2^{-91.00}$ $2^{-75.96}$ [20] 2^{-104} [24]	1h 45m 38s 42s 21h 10m 55s
	22	$2^{-96.00}$ $2^{-96.55}$ [20] 2^{-120} [24]	6h 1m 3h 02s 1day 16h 14m 42s
	23	$2^{-100.00}$ $2^{-115.59}$ [20]	1day 16h 9h 23m

Table 7: Boomerang distinguishers with their corresponding probability and number of rounds for WARP. The computation time is given in hours, minutes and seconds. $(*)^a$ means that we have the score greater or equal than 0.1 while $(*)^b$ that the score is greater or equal than 0.5.

Cipher	Rnds	p	q	r	p^2q^2r	α	Time (in minutes)
WARP	14	$2^{-4.00}$	$2^{-4.00}$	$2^{-4.55}$	$2^{-20.55}$	0	103.85m
	14	$2^{-4.00}$	$2^{-4.00}$	$2^{-4.51}$	$2^{-20.51}$	0.1	26.97m
	14	$2^{-4.00}$	$2^{-4.00}$	$2^{-4.51}$	$2^{-20.51}$	0.5	14.02m
	15	$2^{-4.00}$	$2^{-4.00}$	$2^{-10.50}$	$2^{-26.50}$	0	105.77m
	15	$2^{-4.00}$	$2^{-8.00}$	$2^{-4.51}$	$2^{-28.51}$	0.1	27.60m
	15	$2^{-4.00}$	$2^{-4.00}$	$2^{-10.50}$	$2^{-26.50}$	0.5	14.22m
	16	$2^{-8.00}$	$2^{-8.00}$	$2^{-4.51}$	$2^{-36.51}$	0	104.92m
	16	$2^{-8.00}$	$2^{-8.00}$	$2^{-4.51}$	$2^{-36.51}$	0.1	27.08m
	16	$2^{-8.00}$	$2^{-8.00}$	$2^{-4.51}$	$2^{-36.51}$	0.5	14.57m
	17	$2^{-8.00}$	$2^{-8.00}$	$2^{-10.51}$	$2^{-42.51}$	0	105.98m
	17	$2^{-8.00}$	$2^{-8.00}$	$2^{-10.51}$	$2^{-42.51}$	0.1	26.03m
	17	$2^{-8.00}$	$2^{-8.00}$	$2^{-10.51}$	$2^{-42.51}$	0.5	13.98m
	18	$2^{-8.00}$	$2^{-8.00}$	$2^{-20.00}$	$2^{-52.00}$	0	104.15m
	18	$2^{-8.00}$	$2^{-8.00}$	$2^{-20.19}$	$2^{-52.19}$	0.1	27.33m
	18	$2^{-8.00}$	$2^{-8.00}$	$2^{-20.19}$	$2^{-52.19}$	0.5	14.35m
	19	$2^{-8.00}$	$2^{-14.00}$	$2^{-20.19}$	$2^{-64.19}$	0	105.18m
	19	$2^{-8.00}$	$2^{-14.00}$	$2^{-20.19}$	$2^{-64.19}$	0.1	27.35m
	19	$2^{-8.00}$	$2^{-14.00}$	$2^{-20.19}$	$2^{-64.19}$	0.5	13.27m
	20	$2^{-14.00}$	$2^{-14.00}$	$2^{-20.19}$	$2^{-76.19}$	0	107.50m
	20	$2^{-14.00}$	$2^{-14.00}$	$2^{-20.19}$	$2^{-76.19}$	0.1	27.60m
	20	$2^{-14.00}$	$2^{-14.00}$	$2^{-20.19}$	$2^{-76.19}$	0.5	13.87m
	21	$2^{-14.00}$	$2^{-20.00}$	$2^{-23.00}$	$2^{-91.00}$	0	105.63m
	21	$2^{-14.00}$	$2^{-20.00}$	$2^{-23.00}$	$2^{-91.00}$	0.1	28.60m
	21	$2^{-14.00}$	$2^{-20.00}$	$2^{-23.00}$	$2^{-91.00}$	0.5	14.62m
	22	$2^{-16.00}$	$2^{-20.00}$	$2^{-24.00}$	$2^{-96.00}$	0	361.00m
	22	$2^{-16.00}$	$2^{-20.00}$	$2^{-24.00}$	$2^{-96.00}$	0.1	117.38m
	22	$2^{-16.00}$	$2^{-20.00}$	$2^{-24.00}$	$2^{-96.00}$	0.5	34.55m
	23	$2^{-16.00}$	$2^{-22.00}$	$2^{-24.00}$	$2^{-100.00}$	0	2400.00m
	23	$2^{-16.00}$	$2^{-22.00}$	$2^{-24.00}$	$2^{-100.00}$	0.1	802.00m
	23	—	—	—	—	0.5	3600.00m

Table 8: Results for WARP

13 rounds This paper ($\alpha \geq 0.1$)			
$r_0 = 3, r_m = 8, r_1 = 2, p = 2^{-4.00}, q = 2^{-4.00}, r = 2^{-15.69}, p^2 q^2 r = 2^{-31.69}$			
$\Delta X^{(0)}$	0798000000000000	$\Delta X^{(3)}$	0000000000007000
$\nabla X^{(11)}$	0000000000000080	$\nabla X^{(13)}$	00800000000300f0
13 rounds [20]			
$r_0 = 3, r_m = 8, r_1 = 2, p = 2^{-4}, q = 2^{-4}, r = 2^{-18.32}, p^2 q^2 r = 2^{-34.32}$			
$\Delta X^{(0)}$	0000060052000000	$\Delta X^{(3)}$	0000000000000060
$\nabla X^{(11)}$	6000000000000000	$\nabla X^{(13)}$	2000050000006000
14 rounds This paper ($\alpha \geq 0.1$)			
$r_0 = 3, r_m = 8, r_1 = 3, p = 2^{-4.00}, q = 2^{-4.00}, r = 2^{-24.00}, p^2 q^2 r = 2^{-40.00}$			
$\Delta X^{(0)}$	0d1c000000000000	$\Delta X^{(3)}$	000000000000d000
$\nabla X^{(11)}$	0000000b00000000	$\nabla X^{(13)}$	000d000010b00000
14 rounds [20]			
$r_0 = 3, r_m = 8, r_1 = 3, p = 2^{-4}, q = 2^{-8}, r = 2^{-14.25}, p^2 q^2 r = 2^{-42.45}$			
$\Delta X^{(0)}$	0652000000000000	$\Delta X^{(3)}$	0000000000006000
$\nabla X^{(11)}$	0060000000000000	$\nabla X^{(13)}$	5000060050a00002
15 rounds This paper ($\alpha \geq 0.1$)			
$r_0 = 3, r_m = 8, r_1 = 4, p = 2^{-4.00}, q = 2^{-8.00}, r = 2^{-24.00}, p^2 q^2 r = 2^{-48.00}$			
$\Delta X^{(0)}$	0000000000000f65	$\Delta X^{(3)}$	000000f000000000
$\nabla X^{(11)}$	0000000d00000000	$\nabla X^{(13)}$	001010e00d000c00
15 rounds [20]			
$r_0 = 4, r_m = 8, r_1 = 3, p = 2^{-8}, q = 2^{-8}, r = 2^{-19.03}, p^2 q^2 r = 2^{-51.03}$			
$\Delta X^{(0)}$	052a006500000000	$\Delta X^{(3)}$	0000000060000000
$\nabla X^{(11)}$	6000000000000000	$\nabla X^{(13)}$	a000020000505006
16 rounds This paper ($\alpha \geq 0.1$)			
$r_0 = 4, r_m = 8, r_1 = 4, p = 2^{-8.00}, q = 2^{-8.00}, r = 2^{-23.00}, p^2 q^2 r = 2^{-55.00}$			
$\Delta X^{(0)}$	003f0065000f0000	$\Delta X^{(3)}$	0000000000003000
$\nabla X^{(11)}$	0000000000060000	$\nabla X^{(13)}$	5000060050a00002
16 rounds [20]			
$r_0 = 4, r_m = 8, r_1 = 4, p = 2^{-8}, q = 2^{-8}, r = 2^{-26.04}, p^2 q^2 r = 2^{-58.04}$			
$\Delta X^{(0)}$	052a006500000000	$\Delta X^{(3)}$	0000000060000000
$\nabla X^{(11)}$	0500000000000000	$\nabla X^{(13)}$	70000a0000202005
17 rounds This paper ($\alpha \geq 0.1$)			
$r_0 = 5, r_m = 8, r_1 = 4, p = 2^{-14.00}, q = 2^{-8.00}, r = 2^{-24.00}, p^2 q^2 r = 2^{-68.00}$			
$\Delta X^{(0)}$	005252a706020000	$\Delta X^{(3)}$	0000000060000000
$\nabla X^{(11)}$	0000000f00000000	$\nabla X^{(13)}$	006060200f000500
18 rounds This paper			
$r_0 = 5, r_m = 8, r_1 = 5, p = 2^{-10.00}, q = 2^{-14.00}, r = 2^{-19.14}, p^2 q^2 r = 2^{-67.14}$			
$\Delta X^{(0)}$	000000001c00e40c	$\Delta X^{(3)}$	00010000c0000000
$\nabla X^{(11)}$	0000000000090000	$\nabla X^{(13)}$	306008300f009830
19 rounds This paper			
$r_0 = 5, r_m = 8, r_1 = 6, p = 2^{-14.00}, q = 2^{-16.00}, r = 2^{-24.00}, p^2 q^2 r = 2^{-84.00}$			
$\Delta X^{(0)}$	bd04000000e4e40c	$\Delta X^{(3)}$	c000000000000000
$\nabla X^{(11)}$	00000000ce000000	$\nabla X^{(13)}$	4000ce4040d00e0b

Table 9: Input and output differences found for the boomerang distinguisher of TWINE. If α is present, we cannot find any solutions without the extra filtering.

9 rounds This paper			
$r_0 = 1, r_m = 7, r_1 = 1, p = 2^{-6.00}, q = 2^{-2.00}, r = 2^{-17.44}, p^2 q^2 r = 2^{-33.44}$			
$\Delta X^{(0)}$	0000004004004000	$\Delta X^{(3)}$	0000000000000002
$\nabla X^{(11)}$	0000000000050000	$\nabla X^{(13)}$	0200000002000200
10 rounds This paper			
$r_0 = 2, r_m = 7, r_1 = 1, p = 2^{-8.00}, q = 2^{-2.00}, r = 2^{-24.00}, p^2 q^2 r = 2^{-44.00}$			
$\Delta X^{(0)}$	0000020020000002	$\Delta X^{(3)}$	0a00000000000000
$\nabla X^{(11)}$	0000000000040000	$\nabla X^{(13)}$	0200000002000200
11 rounds This paper			
$r_0 = 2, r_m = 7, r_1 = 2, p = 2^{-8.00}, q = 2^{-8.00}, r = 2^{-24.00}, p^2 q^2 r = 2^{-56.00}$			
$\Delta X^{(0)}$	0000005005001000	$\Delta X^{(3)}$	0060000000000000
$\nabla X^{(11)}$	0000000020000000	$\nabla X^{(13)}$	8880008080008080
12 rounds This paper			
$r_0 = 3, r_m = 7, r_1 = 2, p = 2^{-18.00}, q = 2^{-8.00}, r = 2^{-24.00}, p^2 q^2 r = 2^{-76.00}$			
$\Delta X^{(0)}$	0008088008000800	$\Delta X^{(3)}$	0000000001000000
$\nabla X^{(11)}$	0000000020000000	$\nabla X^{(13)}$	8880008080008080

Table 10: Input and output differences found for the boomerang distinguisher of SKINNY64.

10 rounds our work	
$r_0 = 2, r_m = 7, r_1 = 1, p = 2^{-8.00}, q = 2^{-2.00}, r = 2^{-24.00}, p^2 q^2 r = 2^{-44.00}$	
$\Delta X^{(0)}$	$\Delta X^{(2)}$
00000000000010000010000010000000	00000400000000000000000000000000
$\nabla X^{(12)}$	$\nabla X^{(14)}$
00000000000000002100000000000000	0000200000000000000000200000002000
11 rounds our work	
$r_0 = 2, r_m = 7, r_1 = 2, p = 2^{-8.00}, q = 2^{-8.00}, r = 2^{-24.00}, p^2 q^2 r = 2^{-56.00}$	
$\Delta X^{(0)}$	$\Delta X^{(2)}$
00000000000000100000100000100000	00000004000000000000000000000000
$\nabla X^{(12)}$	$\nabla X^{(14)}$
00000000000000000200000000000000	000202020000000020002000000020002
12 rounds our work	
$r_0 = 3, r_m = 7, r_1 = 2, p = 2^{-22.00}, q = 2^{-8.00}, r = 2^{-24.00}, p^2 q^2 r = 2^{-84.00}$	
$\Delta X^{(0)}$	$\Delta X^{(2)}$
00800000800080008000800080000000	00100000000000000000000000000000
$\nabla X^{(12)}$	$\nabla X^{(14)}$
000000000000000000001000000000	80800080008000000000008000800080
13 rounds our work	
$r_0 = 3, r_m = 7, r_1 = 3, p = 2^{-22.00}, q = 2^{-22.00}, r = 2^{-24.00}, p^2 q^2 r = 2^{-112.00}$	
$\Delta X^{(0)}$	$\Delta X^{(2)}$
00000008080808000008000800000800	00000004000000000000000000000000
$\nabla X^{(12)}$	$\nabla X^{(14)}$
00000000000000002000000000000000	080000080808000000080808080000
14 rounds our work ($\alpha \geq 0.8$)	
$r_0 = 3, r_m = 7, r_1 = 4, p = 2^{-22.00}, q = 2^{-42.00}, r = 2^{-24.00}, p^2 q^2 r = 2^{-152.00}$	
$\Delta X^{(0)}$	$\Delta X^{(2)}$
00800000800080008000800080000000	00100000000000000000000000000000
$\nabla X^{(12)}$	$\nabla X^{(14)}$
000000000000000000001000000000	28000008200008080008080828000800

Table 11: Input and output differences found for the boomerang distinguisher of SKINNY128. If α is present, we cannot find any solutions without the extra filtering.

14 rounds This paper	
$r_0 = 2, r_m = 10, r_1 = 2, p = 2^{-4.00}, q = 2^{-4.00}, r = 2^{-4.55}, p^2 q^2 r = 2^{-20.55}$	
$\Delta X^{(0)}$	$\Delta X^{(2)}$
000000a5000000000000000000a00	00000000000000000000a0000000000000
$\nabla X^{(12)}$	$\nabla X^{(14)}$
000000000000000000a0000000000	0000000005000000000070a000000000
14 rounds [20]	
$r_0 = 2, r_m = 10, r_1 = 2, p = 2^{-4}, q = 2^{-4}, r = 2^{-4.58}, p^2 q^2 r = 2^{-20.58}$	
$\Delta X^{(0)}$	$\Delta X^{(2)}$
000000000000a00000000a500000000	0a0000000000000000000000000000000
$\nabla X^{(12)}$	$\nabla X^{(14)}$
0000a00000000000000000000000000	0000f0a00000000000000000f000000
15 rounds This paper	
$r_0 = 2, r_m = 10, r_1 = 3, p = 2^{-4.00}, q = 2^{-4.00}, r = 2^{-10.50}, p^2 q^2 r = 2^{-26.50}$	
$\Delta X^{(0)}$	$\Delta X^{(2)}$
00000000000000000000a500000a0000	00000000000000a0000000000000000
$\nabla X^{(12)}$	$\nabla X^{(14)}$
0a00000000000000000000000000000	00700000000000000000000000005a000
15 rounds [20]	
$r_0 = 2, r_m = 10, r_1 = 3, p = 2^{-4}, q = 2^{-8}, r = 2^{-4.58}, p^2 q^2 r = 2^{-28.58}$	
$\Delta X^{(0)}$	$\Delta X^{(2)}$
000000000000a00000000a500000000	0a00000000000000000000000000000
$\nabla X^{(12)}$	$\nabla X^{(14)}$
0000a00000000000000000000000000	0a000000f000a00000500a0000000000
16 rounds This paper	
$r_0 = 3, r_m = 10, r_1 = 3, p = 2^{-8.00}, q = 2^{-8.00}, r = 2^{-4.51}, p^2 q^2 r = 2^{-36.51}$	
$\Delta X^{(0)}$	$\Delta X^{(2)}$
00000000000007505a5000000000000	0a00000000000000000000000000000
$\nabla X^{(12)}$	$\nabla X^{(14)}$
0000a00000000000000000000000000	070000005000500000500a0000000000
16 rounds [20]	
$r_0 = 3, r_m = 10, r_1 = 3, p = 2^{-8}, q = 2^{-4}, r = 2^{-10.50}, p^2 q^2 r = 2^{-34.50}$	
$\Delta X^{(0)}$	$\Delta X^{(2)}$
000000000000a00000000aaaa000000	00000000000000000000000000000a
$\nabla X^{(12)}$	$\nabla X^{(14)}$
0000000000000000a00000000000000	0000000000aa0000a00000000000000
17 rounds This paper	
$r_0 = 3, r_m = 10, r_1 = 4, p = 2^{-8.00}, q = 2^{-8.00}, r = 2^{-10.51}, p^2 q^2 r = 2^{-42.51}$	
$\Delta X^{(0)}$	$\Delta X^{(2)}$
000000000000050000000075a5000000	00000000000000000000000000000a
$\nabla X^{(12)}$	$\nabla X^{(14)}$
0000000000000000a00000000000000	500050000000070000000000a000050
18 rounds This paper	
$r_0 = 4, r_m = 10, r_1 = 4, p = 2^{-8.00}, q = 2^{-8.00}, r = 2^{-20.00}, p^2 q^2 r = 2^{-52.00}$	
$\Delta X^{(0)}$	$\Delta X^{(2)}$
a500000500000000000000000007500	0000000000000000000000000000a0
$\nabla X^{(12)}$	$\nabla X^{(14)}$
000000000000000000000000a0000	500a000000075000005000000000000
19 rounds This paper	
$r_0 = 4, r_m = 10, r_1 = 5, p = 2^{-8.00}, q = 2^{-14.00}, r = 2^{-20.19}, p^2 q^2 r = 2^{-64.19}$	
$\Delta X^{(0)}$	$\Delta X^{(2)}$
0000000000007500a500000500000000	000000000000a0000000000000000
$\nabla X^{(12)}$	$\nabla X^{(14)}$
0000000000a0000000000000000000	00000000500007570007070000005a0

Table 12: Input and output differences found for the boomerang distinguisher of WARP from round 14 to round 19.

20 rounds This paper	
$r_0 = 5, r_m = 10, r_1 = 5, p = 2^{-14.00}, q = 2^{-14.00}, r = 2^{-20.19}, p^2 q^2 r = 2^{-76.19}$	
$\Delta X^{(0)}$	$\Delta X^{(2)}$
0000070000000005700575700000a0000	000000000000000a0000000000000000
$\nabla X^{(12)}$	$\nabla X^{(14)}$
00000000000a00000000000000000000	000000000500007570007070000005a0
20 rounds [20]	
$r_0 = 5, r_m = 10, r_1 = 5, p = 2^{-14}, q = 2^{-14}, r = 2^{-19.96}, p^2 q^2 r = 2^{-75.96}$	
$\Delta X^{(0)}$	$\Delta X^{(2)}$
00aa5a00000a000000000a00000000da	000000000000000000000000000000a0
$\nabla X^{(12)}$	$\nabla X^{(14)}$
0000000000000000000000000000a0000	70007050000005a0000000000500007a
21 rounds This paper	
$r_0 = 5, r_m = 10, r_1 = 6, p = 2^{-14.00}, q = 2^{-20.00}, r = 2^{-23.00}, p^2 q^2 r = 2^{-91.00}$	
$\Delta X^{(0)}$	$\Delta X^{(2)}$
0007000000000000570057000a570000	000000a0000000000000000000000000
$\nabla X^{(12)}$	$\nabla X^{(14)}$
000000000000000000000000d00000000	0000707000000005700d007005007070
21 rounds [20]	
$r_0 = 5, r_m = 10, r_1 = 6, p = 2^{-10}, q = 2^{-19}, r = 2^{-26.55}, p^2 q^2 r = 2^{-84.55}$	
$\Delta X^{(0)}$	$\Delta X^{(2)}$
00aa0000000000000000a00000000aa	000000000000000a000000a000000000
$\nabla X^{(12)}$	$\nabla X^{(14)}$
0000000a000000000000000000000000	f00a00a00a00a070000050a000000005
22 rounds This paper	
$r_0 = 6, r_m = 10, r_1 = 6, p = 2^{-16.00}, q = 2^{-20.00}, r = 2^{-24.00}, p^2 q^2 r = 2^{-96.00}$	
$\Delta X^{(0)}$	$\Delta X^{(2)}$
570000a505750000000a000000000000	000000000000000a000000d000000000
$\nabla X^{(12)}$	$\nabla X^{(14)}$
0000000d000000000000000000000000	700d0070050070700000707000000005
22 rounds [20]	
$r_0 = 6, r_m = 10, r_1 = 6, p = 2^{-16}, q = 2^{-19}, r = 2^{-26.55}, p^2 q^2 r = 2^{-96.55}$	
$\Delta X^{(0)}$	$\Delta X^{(2)}$
5a0000aa05a50000000a000000000000	000000000000000a000000a000000000
$\nabla X^{(12)}$	$\nabla X^{(14)}$
0000000a000000000000000000000000	000000a0000000000000000000000000
23 rounds This paper	
$r_0 = 6, r_m = 10, r_1 = 7, p = 2^{-16.00}, q = 2^{-22.00}, r = 2^{-24.00}, p^2 q^2 r = 2^{-100.00}$	
$\Delta X^{(0)}$	$\Delta X^{(2)}$
0075a500000f000000000500000000a5	000000f000000000000000000a00000
$\nabla X^{(12)}$	$\nabla X^{(14)}$
0000000000000af0000000000000000	000070a000000005500a00700500a070
23 rounds [20]	
$r_0 = 6, r_m = 10, r_1 = 7, p = 2^{-24.00}, q = 2^{-20.00}, r = 2^{-27.59}, p^2 q^2 r = 2^{-115.59}$	
$\Delta X^{(0)}$	$\Delta X^{(2)}$
00000000aaaaaaa0a00aa00000a000a	a0000000000000000000000000000000
$\nabla X^{(12)}$	$\nabla X^{(14)}$
0000000000000aa0000000000000000	0000a0a00000000aa00a00a00a0a0

Table 13: Input and output differences found for the boomerang distinguisher of WARP from round 20 to 23.